

-

[barter_integration](#)0.5.3

[Validator](#)

[Required Methods](#)

- [validate](#)

[Implementors](#)

[In crate barter_integration](#)

[?](#)
[Change settings](#)

Trait [barter_integration::Validator](#)

Copy item path

[source](#) ·

[-]

```
pub trait Validator {  
    // Required method  
    fn validate(self) -> Result<Self, SocketError>  
    where Self: Sized;  
}
```

Expand description

[Validator](#)s are capable of determining if their internal state is satisfactory to fulfill some use case defined by the implementor.

Required Methods§

[source](#)

```
fn validate(self) -> Result<Self, SocketError>  
where Self: Sized,
```

Check if Self is valid for some use case.

Implementors§

[barter_integration](#)0.5.3

- [All Items](#)
- [Modules](#)
- [Structs](#)
- [Traits](#)

?

[Change settings](#)

Crate [barter_integration](#)

Copy item path

[source](#) ·

[−]

Expand description

§Barter-Integration

High-performance, low-level framework for composing flexible web integrations.

Utilised by other Barter trading ecosystem crates to build robust financial exchange integrations, primarily for public data collection & trade execution. It is:

- **Low-Level:** Translates raw data streams communicated over the web into any desired data model using arbitrary data transformations.
- **Flexible:** Compatible with any protocol (WebSocket, FIX, Http, etc.), any input/output model, and any user defined transformations.

§Core abstractions:

- **RestClient** providing configurable signed Http communication between client & server.
- **ExchangeStream** providing configurable communication over any asynchronous stream protocols (WebSocket, FIX, etc.).

Both core abstractions provide the robust glue you need to conveniently translate between server & client data models.

Modules§

- [de](#)
Utilities to assist deserialisation.
- [error](#)
All [Errors](#) generated in Barter-Integration.
- [metric](#)
Contains the flexible `Metric` type used for representing real-time metrics generically.
- [model](#)
Foundational data structures that define the building blocks used by the rest of the Barter ecosystem.
- [protocol](#)
Contains `StreamParser` implementations for transforming communication protocol specific messages into a generic output data structure.

Structs§

- [ExchangeStream](#)
An [ExchangeStream](#) is a communication protocol agnostic [Stream](#). It polls protocol messages from the inner [Stream](#), and transforms them into the desired output data structure.

Traits§

- [Transformer](#)
[Transformers](#) are capable of transforming any `Input` into an iterator of `Result<Self::Output, Self::Error>s`.
- [Validator](#)
[Validators](#) are capable of determining if their internal state is satisfactory to fulfill some use case defined by the implementor.



[barter_integration](#)0.5.3

[Transformer](#)

[Required Associated Types](#)

- [Error](#)
- [Input](#)
- [Output](#)
- [OutputIter](#)

Required Methods

- [transform](#)

Implementors

In crate [barter_integration](#)

[?](#)

[Change settings](#)

Trait [barter_integration::Transformer](#)

Copy item path

[source](#) ·

[−]

```
pub trait Transformer {
    type Error;
    type Input: for<'de> Deserialize<'de>;
    type Output;
    type OutputIter: IntoIterator<Item = Result<Self::Output, Self::Error>>;

    // Required method
    fn transform(&mut self, input: Self::Input) -> Self::OutputIter;
}
```

Expand description

[Transformers](#) are capable of transforming any Input into an iterator of Result<Self::Output, Self::Error>s.

Required Associated Types§

[source](#)

type [Error](#)

[source](#)

type [Input](#): for<'de> [Deserialize](#)<'de>

[source](#)

type [Output](#)

[source](#)

```
type OutputIter: IntoIterator<Item = Result<Self::Output,  
Self::Error>>
```

Required Methods§

[source](#)

```
fn transform(&mut self, input: Self::Input) -> Self::OutputIter
```

Implementors§



[barter_integration](#)0.5.3

[ExchangeStream](#)

[Fields](#)

- [buffer](#)
- [protocol_marker](#)
- [stream](#)
- [transformer](#)

[Methods](#)

- [new](#)

[Trait Implementations](#)

- [Debug](#)
- [Stream](#)
- [Unpin](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)

- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [StreamExt](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [TryStream](#)
- [TryStreamExt](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In crate barter_integration](#)

[?](#)

[Change settings](#)

Struct [barter_integration::ExchangeStream](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct ExchangeStream<Protocol, InnerStream, StreamTransformer>
where
    Protocol: StreamParser,
    InnerStream: Stream,
    StreamTransformer: Transformer,
{
    pub stream: InnerStream,
    pub transformer: StreamTransformer,
    pub buffer: VecDeque<Result<StreamTransformer::Output, StreamTransformer::Error>>,
    pub protocol_marker: PhantomData<Protocol>,
}
```

Expand description

An [ExchangeStream](#) is a communication protocol agnostic [Stream](#). It polls protocol messages from the inner [Stream](#), and transforms them into the desired output data structure.

Fields§

[stream](#): InnerStream[transformer](#): StreamTransformer[buffer](#): [VecDeque](#)<[Result](#)<StreamTransformer::Output, StreamTransformer::Error>>[protocol_marker](#): [PhantomData](#)<Protocol>

Implementations§

[source§](#)

```
impl<Protocol, InnerStream, StreamTransformer>  
ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where Protocol: StreamParser, InnerStream: Stream,  
StreamTransformer: Transformer,
```

[source](#)

```
pub fn new(stream: InnerStream, transformer: StreamTransformer) -  
> Self
```

Trait Implementations§

[source§](#)

```
impl<Protocol, InnerStream, StreamTransformer> Debug  
for ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where Protocol: StreamParser + Debug, InnerStream:  
Stream + Debug, StreamTransformer: Transformer +  
Debug, StreamTransformer::Output: Debug,  
StreamTransformer::Error: Debug,
```

[source§](#)

```
fn fmt(&self, f: &mut Formatter<'_>) -> Result
```

Formats the value using the given formatter. [Read more](#)

[source§](#)

```
impl<Protocol, InnerStream, StreamTransformer> Stream  
for ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where Protocol: StreamParser, InnerStream: Stream<Item  
= Result<Protocol::Message, Protocol::Error>> + Unpin,  
StreamTransformer: Transformer,  
StreamTransformer::Error: From<SocketError>,
```

§

```
type Item = Result<<StreamTransformer as Transformer>::Output,  
<StreamTransformer as Transformer>::Error>
```

Values yielded by the stream.

[source§](#)

```
fn poll_next( self: Pin<&mut Self>, cx: &mut Context<'_> ) ->  
Poll<Option<Self::Item>>
```

Attempt to pull out the next value of this stream, registering the current task for wakeup if the value is not yet available, and returning None if the stream is exhausted. [Read more](#)

[source§](#)

```
fn size_hint(&self) -> (usize, Option<usize>)
```

Returns the bounds on the remaining length of the stream. [Read more](#)

[source§](#)

```
impl<'pin, Protocol, InnerStream, StreamTransformer>  
Unpin for ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where Protocol: StreamParser, InnerStream: Stream,  
StreamTransformer: Transformer, __ExchangeStream<'pin,  
Protocol, InnerStream, StreamTransformer>: Unpin,
```

Auto Trait Implementations§

§

```
impl<Protocol, InnerStream, StreamTransformer>  
RefUnwindSafe for ExchangeStream<Protocol,  
InnerStream, StreamTransformer>  
where InnerStream: RefUnwindSafe, Protocol:  
RefUnwindSafe, StreamTransformer: RefUnwindSafe,  
<StreamTransformer as Transformer>::Error:  
RefUnwindSafe, <StreamTransformer as  
Transformer>::Output: RefUnwindSafe,
```

§

```
impl<Protocol, InnerStream, StreamTransformer> Send  
for ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where InnerStream: Send, Protocol: Send,  
StreamTransformer: Send, <StreamTransformer as  
Transformer>::Error: Send, <StreamTransformer as  
Transformer>::Output: Send,
```

[§](#)

```
impl<Protocol, InnerStream, StreamTransformer> Sync for  
ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where InnerStream: Sync, Protocol: Sync,  
StreamTransformer: Sync, <StreamTransformer as  
Transformer>::Error: Sync, <StreamTransformer as  
Transformer>::Output: Sync,
```

[§](#)

```
impl<Protocol, InnerStream, StreamTransformer>  
UnwindSafe for ExchangeStream<Protocol, InnerStream,  
StreamTransformer>  
where InnerStream: UnwindSafe, Protocol: UnwindSafe,  
StreamTransformer: UnwindSafe, <StreamTransformer as  
Transformer>::Error: UnwindSafe, <StreamTransformer as  
Transformer>::Output: UnwindSafe,
```

Blanket Implementations[§](#)

[source](#)[§](#)

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)[§](#)

```
fn type\_id(&self) -> TypeId
```

Gets the TypeId of self. [Read more](#)
[source](#)[§](#)

```
impl<T> Borrow<T> for T  
where T: ?Sized,
```

[source](#)[§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From<T>](#) for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [StreamExt](#) for T
where T: [Stream](#) + ?[Sized](#),**

[source§](#)

**fn [next](#)(&mut self) -> [Next](#)<'_, Self>
where Self: [Unpin](#),**

Creates a future that resolves to the next item in the stream. [Read more](#)
[source§](#)

**fn [into_future](#)(self) -> [StreamFuture](#)<Self>
where Self: [Sized](#) + [Unpin](#),**

Converts this stream into a future of (next_item, tail_of_stream). If the stream terminates, then the next item is [None](#). [Read more](#)
[source§](#)

**fn [map](#)<T, F>(self, f: F) -> [Map](#)<Self, F>
where F: [FnMut](#)(Self::Item) -> T, Self: [Sized](#),**

Maps this stream's items to a different type, returning a new stream of the resulting type. [Read more](#)
[source§](#)

**fn [enumerate](#)(self) -> [Enumerate](#)<Self>
where Self: [Sized](#),**

Creates a stream which gives the current iteration count as well as the next value. [Read more](#)
[source§](#)

```
fn filter<Fut, F>(self, f: F) -> Filter<Self, Fut, F>  
where F: FnMut(&Self::Item) -> Fut, Fut: Future<Output = bool>,  
Self: Sized,
```

Filters the values produced by this stream according to the provided asynchronous predicate. [Read more](#)
[source§](#)

```
fn filter_map<Fut, T, F>(self, f: F) -> FilterMap<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future<Output =  
Option<T>>, Self: Sized,
```

Filters the values produced by this stream while simultaneously mapping them to a different type according to the provided asynchronous closure. [Read more](#)
[source§](#)

```
fn then<Fut, F>(self, f: F) -> Then<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future, Self: Sized,
```

Computes from this stream's items new items of a different type using an asynchronous closure. [Read more](#)
[source§](#)

```
fn collect<C>(self) -> Collect<Self, C>  
where C: Default + Extend<Self::Item>, Self: Sized,
```

Transforms a stream into a collection, returning a future representing the result of that computation. [Read more](#)
[source§](#)

```
fn unzip<A, B, FromA, FromB>(self) -> Unzip<Self, FromA, FromB>  
where FromA: Default + Extend<A>, FromB: Default + Extend<B>,  
Self: Sized + Stream<Item = (A, B)>,
```

Converts a stream of pairs into a future, which resolves to pair of containers. [Read more](#)
[source§](#)

```
fn concat(self) -> Concat<Self>  
where Self: Sized, Self::Item: Extend<<Self::Item as  
IntoIterator>::Item> + IntoIterator + Default,
```

Concatenate all items of a stream into a single extendable destination, returning a future representing the end result. [Read more](#)
[source§](#)

```
fn count(self) -> Count<Self>  
where Self: Sized,
```

Drives the stream to completion, counting the number of items. [Read more source§](#)

```
fn cycle(self) -> Cycle<Self>  
where Self: Sized + Clone,
```

Repeats a stream endlessly. [Read more source§](#)

```
fn fold<T, Fut, F>(self, init: T, f: F) -> Fold<Self, Fut, T, F>  
where F: FnMut(T, Self::Item) -> Fut, Fut: Future<Output = T>, Self:  
Sized,
```

Execute an accumulating asynchronous computation over a stream, collecting all the values into one final result. [Read more source§](#)

```
fn any<Fut, F>(self, f: F) -> Any<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future<Output = bool>, Self:  
Sized,
```

Execute predicate over asynchronous stream, and return true if any element in stream satisfied a predicate. [Read more source§](#)

```
fn all<Fut, F>(self, f: F) -> All<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future<Output = bool>, Self:  
Sized,
```

Execute predicate over asynchronous stream, and return true if all element in stream satisfied a predicate. [Read more source§](#)

```
fn flatten(self) -> Flatten<Self>  
where Self::Item: Stream, Self: Sized,
```

Flattens a stream of streams into just one continuous stream. [Read more source§](#)

```
fn flatten\_unordered( self, limit: impl Into<Option<usize>> ) ->  
FlattenUnorderedWithFlowController<Self, Q>  
where Self::Item: Stream + Unpin, Self: Sized,
```

Flattens a stream of streams into just one continuous stream. Polls inner streams produced by the base stream concurrently. [Read more source§](#)

```
fn flat_map<U, F>(self, f: F) -> FlatMap<Self, U, F>  
where F: FnMut(Self::Item) -> U, U: Stream, Self: Sized,
```

Maps a stream like [StreamExt::map](#) but flattens nested Streams. [Read more source§](#)

```
fn flat_map_unordered<U, F>( self, limit: impl Into<Option<usize>>,  
f: F ) -> FlatMapUnordered<Self, U, F>  
where U: Stream + Unpin, F: FnMut(Self::Item) -> U, Self: Sized,
```

Maps a stream like [StreamExt::map](#) but flattens nested Streams and polls them concurrently, yielding items in any order, as they made available. [Read more source§](#)

```
fn scan<S, B, Fut, F>(self, initial_state: S, f: F) -> Scan<Self, S, Fut, F>  
where F: FnMut(&mut S, Self::Item) -> Fut, Fut: Future<Output = Option<B>>, Self: Sized,
```

Combinator similar to [StreamExt::fold](#) that holds internal state and produces a new stream. [Read more source§](#)

```
fn skip_while<Fut, F>(self, f: F) -> SkipWhile<Self, Fut, F>  
where F: FnMut(&Self::Item) -> Fut, Fut: Future<Output = bool>, Self: Sized,
```

Skip elements on this stream while the provided asynchronous predicate resolves to true. [Read more source§](#)

```
fn take_while<Fut, F>(self, f: F) -> TakeWhile<Self, Fut, F>  
where F: FnMut(&Self::Item) -> Fut, Fut: Future<Output = bool>, Self: Sized,
```

Take elements from this stream while the provided asynchronous predicate resolves to true. [Read more source§](#)

```
fn take_until<Fut>(self, fut: Fut) -> TakeUntil<Self, Fut>  
where Fut: Future, Self: Sized,
```

Take elements from this stream until the provided future resolves. [Read more source§](#)

```
fn for_each<Fut, F>(self, f: F) -> ForEach<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future<Output = Q>, Self: Sized,
```

Runs this stream to completion, executing the provided asynchronous closure for each element on the stream. [Read more](#)
[source§](#)

```
fn for_each_concurrent<Fut, F>( self, limit: impl  
Into<Option<usize>>, f: F ) -> ForEachConcurrent<Self, Fut, F>  
where F: FnMut(Self::Item) -> Fut, Fut: Future<Output = Q>, Self: Sized,
```

Runs this stream to completion, executing the provided asynchronous closure for each element on the stream concurrently as elements become available. [Read more](#)
[source§](#)

```
fn take(self, n: usize) -> Take<Self>  
where Self: Sized,
```

Creates a new stream of at most n items of the underlying stream. [Read more](#)
[source§](#)

```
fn skip(self, n: usize) -> Skip<Self>  
where Self: Sized,
```

Creates a new stream which skips n items of the underlying stream. [Read more](#)
[source§](#)

```
fn fuse(self) -> Fuse<Self>  
where Self: Sized,
```

Fuse a stream such that `poll_next` will never again be called once it has finished. This method can be used to turn any Stream into a FusedStream. [Read more](#)
[source§](#)

```
fn by_ref(&mut self) -> &mut Self
```

Borrows a stream, rather than consuming it. [Read more](#)
[source§](#)

```
fn catch_unwind(self) -> CatchUnwind<Self>  
where Self: Sized + UnwindSafe,
```

Catches unwinding panics while polling the stream. [Read more](#)
[source§](#)

```
fn boxed<'a>(self) -> Pin<Box<dyn Stream<Item = Self::Item> +  
Send + 'a>>  
where Self: Sized + Send + 'a,
```

Wrap the stream in a Box, pinning it. [Read more source§](#)

```
fn boxed_local<'a>(self) -> Pin<Box<dyn Stream<Item = Self::Item>  
+ 'a>>  
where Self: Sized + 'a,
```

Wrap the stream in a Box, pinning it. [Read more source§](#)

```
fn buffered(self, n: usize) -> Buffered<Self>  
where Self::Item: Future, Self: Sized,
```

An adaptor for creating a buffered list of pending futures. [Read more source§](#)

```
fn buffer_unordered(self, n: usize) -> BufferUnordered<Self>  
where Self::Item: Future, Self: Sized,
```

An adaptor for creating a buffered list of pending futures (unordered). [Read more source§](#)

```
fn zip<St>(self, other: St) -> Zip<Self, St>  
where St: Stream, Self: Sized,
```

An adaptor for zipping two streams together. [Read more source§](#)

```
fn chain<St>(self, other: St) -> Chain<Self, St>  
where St: Stream<Item = Self::Item>, Self: Sized,
```

Adapter for chaining two streams. [Read more source§](#)

```
fn peekable(self) -> Peekable<Self>  
where Self: Sized,
```

Creates a new stream which exposes a peek method. [Read more source§](#)

```
fn chunks(self, capacity: usize) -> Chunks<Self>  
where Self: Sized,
```

An adaptor for chunking up items of the stream inside a vector. [Read more](#)

[source§](#)

```
fn ready\_chunks(self, capacity: usize) -> ReadyChunks<Self>  
where Self: Sized,
```

An adaptor for chunking up ready items of the stream inside a vector. [Read more](#)
[source§](#)

```
fn forward<S>(self, sink: S) -> Forward<Self, S>  
where S: Sink<Self::Ok, Error = Self::Error>, Self: TryStream +  
Sized,
```

A future that completes after the given stream has been fully processed into the sink and the sink has been flushed and closed. [Read more](#)
[source§](#)

```
fn split<Item>(self) -> (SplitSink<Self, Item>, SplitStream<Self>)  
where Self: Sink<Item> + Sized,
```

Splits this Stream + Sink object into separate Sink and Stream objects.
[Read more](#)
[source§](#)

```
fn inspect<F>(self, f: F) -> Inspect<Self, F>  
where F: FnMut(&Self::Item), Self: Sized,
```

Do something with each item of this stream, afterwards passing it on. [Read more](#)
[source§](#)

```
fn left\_stream<B>(self) -> Either<Self, B>  
where B: Stream<Item = Self::Item>, Self: Sized,
```

Wrap this stream in an Either stream, making it the left-hand variant of that Either. [Read more](#)
[source§](#)

```
fn right\_stream<B>(self) -> Either<B, Self>  
where B: Stream<Item = Self::Item>, Self: Sized,
```

Wrap this stream in an Either stream, making it the right-hand variant of that Either. [Read more](#)
[source§](#)

```
fn poll_next_unpin(&mut self, cx: &mut Context<'_>) ->  
Poll<Option<Self::Item>>  
where Self: Unpin,
```

A convenience method for calling [Stream::poll_next](#) on [Unpin](#) stream types.

[source](#)

```
fn select_next_some(&mut self) -> SelectNextSome<'_, Self>  
where Self: Unpin + FusedStream,
```

Returns a [Future](#) that resolves when the next item in this stream is ready.

[Read more](#)

[source](#)

```
impl<T, U> TryFrom<U> for T  
where U: Into<T>,
```

§

```
type Error = Infallible
```

The type returned in the event of a conversion error.

[source](#)

```
fn try_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source](#)

```
impl<T, U> TryInto<U> for T  
where U: TryFrom<T>,
```

§

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source](#)

```
impl<S, T, E> TryStream for S  
where S: Stream<Item = Result<T, E>> + ?Sized,
```

§

type [Ok](#) = T

The type of successful values yielded by this future
[§](#)

type [Error](#) = E

The type of failures yielded by this future
[source§](#)

```
fn try\_poll\_next( self: Pin<&mut S>, cx: &mut Context<'_> ) ->  
Poll<Option<Result<<S as TryStream>::Ok, <S as  
TryStream>::Error>>>
```

Poll this TryStream as if it were a Stream. [Read more](#)
[source§](#)

**impl<[S](#)> [TryStreamExt](#) for [S](#)
where [S](#): [TryStream](#) + ?[Sized](#),**

[source§](#)

```
fn err\_into<E>(self) -> ErrInto<Self, E>  
where Self: Sized, Self::Error: Into<E>,
```

Wraps the current stream in a new stream which converts the error type into the one provided. [Read more](#)
[source§](#)

```
fn map\_ok<T, F>(self, f: F) -> MapOk<Self, F>  
where Self: Sized, F: FnMut(Self::Ok) -> T,
```

Wraps the current stream in a new stream which maps the success value using the provided closure. [Read more](#)
[source§](#)

```
fn map\_err<E, F>(self, f: F) -> MapErr<Self, F>  
where Self: Sized, F: FnMut(Self::Error) -> E,
```

Wraps the current stream in a new stream which maps the error value using the provided closure. [Read more](#)
[source§](#)

```
fn and\_then<Fut, F>(self, f: F) -> AndThen<Self, Fut, F>  
where F: FnMut(Self::Ok) -> Fut, Fut: TryFuture<Error =  
Self::Error>, Self: Sized,
```

Chain on a computation for when a value is ready, passing the successful results to the provided closure f. [Read more](#)

[source§](#)

```
fn or_else<Fut, F>(self, f: F) -> OrElse<Self, Fut, F>  
where F: FnMut(Self::Error) -> Fut, Fut: TryFuture<Ok = Self::Ok>,  
Self: Sized,
```

Chain on a computation for when an error happens, passing the erroneous result to the provided closure f. [Read more](#)
[source§](#)

```
fn inspect_ok<F>(self, f: F) -> InspectOk<Self, F>  
where F: FnMut(&Self::Ok), Self: Sized,
```

Do something with the success value of this stream, afterwards passing it on. [Read more](#)
[source§](#)

```
fn inspect_err<F>(self, f: F) -> InspectErr<Self, F>  
where F: FnMut(&Self::Error), Self: Sized,
```

Do something with the error value of this stream, afterwards passing it on. [Read more](#)
[source§](#)

```
fn into_stream(self) -> IntoStream<Self>  
where Self: Sized,
```

Wraps a [TryStream](#) into a type that implements [Stream](#) [Read more](#)
[source§](#)

```
fn try_next(&mut self) -> TryNext<'_, Self>  
where Self: Unpin,
```

Creates a future that attempts to resolve the next item in the stream. If an error is encountered before the next item, the error is returned instead. [Read more](#)
[source§](#)

```
fn try_for_each<Fut, F>(self, f: F) -> TryForEach<Self, Fut, F>  
where F: FnMut(Self::Ok) -> Fut, Fut: TryFuture<Ok = (), Error = Self::Error>,  
Self: Sized,
```

Attempts to run this stream to completion, executing the provided asynchronous closure for each element on the stream. [Read more](#)
[source§](#)

```
fn try_skip_while<Fut, F>(self, f: F) -> TrySkipWhile<Self, Fut, F>  
where F: FnMut(&Self::Ok) -> Fut, Fut: TryFuture<Ok = bool, Error =  
Self::Error>, Self: Sized,
```

Skip elements on this stream while the provided asynchronous predicate resolves to true. [Read more](#)
[source](#)

```
fn try_take_while<Fut, F>(self, f: F) -> TryTakeWhile<Self, Fut, F>  
where F: FnMut(&Self::Ok) -> Fut, Fut: TryFuture<Ok = bool, Error =  
Self::Error>, Self: Sized,
```

Take elements on this stream while the provided asynchronous predicate resolves to true. [Read more](#)
[source](#)

```
fn try_for_each_concurrent<Fut, F>( self, limit: impl  
Into<Option<usize>>, f: F ) -> TryForEachConcurrent<Self, Fut, F>  
where F: FnMut(Self::Ok) -> Fut, Fut: Future<Output = Result<(),  
Self::Error>>, Self: Sized,
```

Attempts to run this stream to completion, executing the provided asynchronous closure for each element on the stream concurrently as elements become available, exiting as soon as an error occurs. [Read more](#)
[source](#)

```
fn try_collect<C>(self) -> TryCollect<Self, C>  
where C: Default + Extend<Self::Ok>, Self: Sized,
```

Attempt to transform a stream into a collection, returning a future representing the result of that computation. [Read more](#)
[source](#)

```
fn try_chunks(self, capacity: usize) -> TryChunks<Self>  
where Self: Sized,
```

An adaptor for chunking up successful items of the stream inside a vector. [Read more](#)
[source](#)

```
fn try_ready_chunks(self, capacity: usize) -> TryReadyChunks<Self>  
where Self: Sized,
```

An adaptor for chunking up successful, ready items of the stream inside a vector. [Read more](#)
[source](#)

```
fn try\_filter<Fut, F>(self, f: F) -> TryFilter<Self, Fut, F>
where Fut: Future<Output = bool>, F: FnMut(&Self::Ok) -> Fut, Self:
Sized,
```

Attempt to filter the values produced by this stream according to the provided asynchronous closure. [Read more](#)
[source§](#)

```
fn try\_filter\_map<Fut, F, T>(self, f: F) -> TryFilterMap<Self, Fut, F>
where Fut: TryFuture<Ok = Option<T>, Error = Self::Error>, F:
FnMut(Self::Ok) -> Fut, Self: Sized,
```

Attempt to filter the values produced by this stream while simultaneously mapping them to a different type according to the provided asynchronous closure. [Read more](#)
[source§](#)

```
fn try\_flatten\_unordered( self, limit: impl Into<Option<usize>> ) ->
TryFlattenUnordered<Self>
where Self::Ok: TryStream + Unpin, <Self::Ok as TryStream>::Error:
From<Self::Error>, Self: Sized,
```

Flattens a stream of streams into just one continuous stream. Produced streams will be polled concurrently and any errors will be passed through without looking at them. If the underlying base stream returns an error, it will be **immediately** propagated. [Read more](#)
[source§](#)

```
fn try\_flatten(self) -> TryFlatten<Self>
where Self::Ok: TryStream, <Self::Ok as TryStream>::Error:
From<Self::Error>, Self: Sized,
```

Flattens a stream of streams into just one continuous stream. [Read more](#)
[source§](#)

```
fn try\_fold<T, Fut, F>(self, init: T, f: F) -> TryFold<Self, Fut, T, F>
where F: FnMut(T, Self::Ok) -> Fut, Fut: TryFuture<Ok = T, Error =
Self::Error>, Self: Sized,
```

Attempt to execute an accumulating asynchronous computation over a stream, collecting all the values into one final result. [Read more](#)
[source§](#)

```
fn try\_concat(self) -> TryConcat<Self>
where Self: Sized, Self::Ok: Extend<<Self::Ok as IntoIterator>::Item>
+ IntoIterator + Default,
```

Attempt to concatenate all items of a stream into a single extendable destination, returning a future representing the end result. [Read more](#)
[source§](#)

```
fn try_buffer_unordered(self, n: usize) -> TryBufferUnordered<Self>  
where Self::Ok: TryFuture<Error = Self::Error>, Self: Sized,
```

Attempt to execute several futures from a stream concurrently (unordered).
[Read more](#)
[source§](#)

```
fn try_buffered(self, n: usize) -> TryBuffered<Self>  
where Self::Ok: TryFuture<Error = Self::Error>, Self: Sized,
```

Attempt to execute several futures from a stream concurrently. [Read more](#)
[source§](#)

```
fn try_poll_next_unpin( &mut self, cx: &mut Context<'_> ) ->  
Poll<Option<Result<Self::Ok, Self::Error>>>  
where Self: Unpin,
```

A convenience method for calling `TryStream::try_poll_next` on `Unpin` stream types.
[source§](#)

```
fn into_async_read(self) -> IntoAsyncRead<Self>  
where Self: Sized + TryStreamExt<Error = Error>, Self::Ok:  
AsRef<[u8]>,
```

Adapter that converts this stream into an `AsyncBufRead`. [Read more](#)
[source§](#)

```
fn try_all<Fut, F>(self, f: F) -> TryAll<Self, Fut, F>  
where Self: Sized, F: FnMut(Self::Ok) -> Fut, Fut: Future<Output =  
bool>,
```

Attempt to execute a predicate over an asynchronous stream and evaluate if all items satisfy the predicate. Exits early if an Err is encountered or if an Ok item is found that does not satisfy the predicate. [Read more](#)
[source§](#)

```
fn try_any<Fut, F>(self, f: F) -> TryAny<Self, Fut, F>  
where Self: Sized, F: FnMut(Self::Ok) -> Fut, Fut: Future<Output =  
bool>,
```

Attempt to execute a predicate over an asynchronous stream and evaluate if any items satisfy the predicate. Exits early if an Err is encountered or if an Ok item is found that satisfies the predicate. [Read more](#)
[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[barter_integration](#)0.5.3

- [Structs](#)
- [Enums](#)
- [Traits](#)
- [Functions](#)
- [Type Aliases](#)

[?](#)
[Change settings](#)

List of all items

Structs

- [ExchangeStream](#)
- [metric::Field](#)
- [metric::Metric](#)
- [metric::Tag](#)
- [model::Exchange](#)
- [model::Market](#)
- [model::MarketId](#)
- [model::SubscriptionId](#)
- [model::instrument::Instrument](#)
- [model::instrument::kind::FutureContract](#)
- [model::instrument::kind::OptionContract](#)
- [model::instrument::symbol::Symbol](#)

- [protocol::http::private::RequestSigner](#)
- [protocol::http::private::encoder::HexEncoder](#)
- [protocol::http::public::PublicNoHeaders](#)
- [protocol::http::rest::client::RestClient](#)
- [protocol::websocket::WebSocketParser](#)

Enums

- [error::SocketError](#)
- [metric::Value](#)
- [model::Side](#)
- [model::instrument::kind::InstrumentKind](#)
- [model::instrument::kind::OptionExercise](#)
- [model::instrument::kind::OptionKind](#)

Traits

- [Transformer](#)
- [Validator](#)
- [protocol::StreamParser](#)
- [protocol::http::BuildStrategy](#)
- [protocol::http::HttpParser](#)
- [protocol::http::private::Signer](#)
- [protocol::http::private::encoder::Encoder](#)
- [protocol::http::rest::RestRequest](#)

Functions

- [de::datetime_utc_from_epoch_duration](#)
- [de::de_str](#)
- [de::de_str_f64_epoch_ms_as_datetime_utc](#)
- [de::de_str_f64_epoch_s_as_datetime_utc](#)
- [de::de_str_u64_epoch_ms_as_datetime_utc](#)
- [de::de_u64_epoch_ms_as_datetime_utc](#)
- [de::extract_next](#)
- [de::se_element_to_vector](#)
- [protocol::websocket::connect](#)
- [protocol::websocket::is_websocket_disconnected](#)
- [protocol::websocket::process_binary](#)
- [protocol::websocket::process_close_frame](#)
- [protocol::websocket::process_frame](#)
- [protocol::websocket::process_ping](#)
- [protocol::websocket::process_pong](#)
- [protocol::websocket::process_text](#)

Type Aliases

- [protocol::websocket::WebSocket](#)
- [protocol::websocket::WsError](#)
- [protocol::websocket::WsMessage](#)
- [protocol::websocket::WsSink](#)

- [protocol::websocket::WsStream](#)

protocol

[barter_integration](#)0.5.3

[Module protocol](#)

- [Modules](#)
- [Traits](#)

[In crate barter_integration](#)

[?](#)
[Change settings](#)

Module [barter_integration::protocol](#)

[source](#) ·

Expand description

Contains StreamParser implementations for transforming communication protocol specific messages into a generic output data structure.

Modules§

- [http](#)
Contains HTTP client capable of executing signed & unsigned requests, as well as an associated exchange oriented HTTP request.
- [websocket](#)
Contains useful WebSocket type aliases and a default WebSocket implementation of a [StreamParser](#).

Traits§

- [StreamParser](#)
StreamParsers are capable of parsing the input messages from a given stream protocol (eg/ WebSocket, Financial Information eXchange (FIX), etc.) and deserialising into an Output.

[barter_integration](#)0.5.3

[StreamParser](#)

[Required Associated Types](#)

- [Error](#)
- [Message](#)
- [Stream](#)

[Required Methods](#)

- [parse](#)

[Object Safety](#)

[Implementors](#)

[In `barter\_integration::protocol`](#)

[?](#)
[Change settings](#)

Trait [barter_integration::protocol::](#)

[StreamParser](#)

Copy item path

[source](#) ·

[–]

```
pub trait StreamParser {
  type Stream: Stream;
  type Message;
  type Error;

  // Required method
  fn parse<Output>(
    input: Result<Self::Message, Self::Error>
  ) -> Option<Result<Output, SocketError>>
    where Output: DeserializeOwned;
}
```

Expand description

StreamParsers are capable of parsing the input messages from a given stream protocol (eg/ WebSocket, Financial Information eXchange (FIX), etc.) and deserialising into an Output.

Required Associated Types§

[source](#)

type **Stream**: [Stream](#)

[source](#)

type **Message**

[source](#)

type **Error**

Required Methods§

[source](#)

```
fn parse<Output>( input: Result<Self::Message, Self::Error> ) ->  
Option<Result<Output, SocketError>>  
where Output: DeserializeOwned,
```

Object Safety§

This trait is **not** [object safe](#).

Implementors§

[source](#)§

impl [StreamParser](#) for [WebSocketParser](#)

§

type [Stream](#) = [WebSocketStream](#)<[MaybeTlsStream](#)<[TcpStream](#)>>

§

type [Message](#) = [Message](#)

§

type [Error](#) = [Error](#)

protocol - http

[barter_integration](#)0.5.3

[Module http](#)

- [Modules](#)
- [Traits](#)

[In barter_integration::protocol](#)

[?](#)

[Change settings](#)

Module [barter_integration::protocol::http](#)

Copy item path

[source](#) · [\[-\]](#)

Expand description

Contains HTTP client capable of executing signed & unsigned requests, as well as an associated exchange oriented HTTP request.

Modules§

- [private](#)
Defines a configurable [RequestSigner](#) that signs Http [RestRequest](#) using API specific logic.
- [public](#)
Defines a default [BuildStrategy](#) that builds a non-authenticated Http [RestRequest](#) with no headers.
- [rest](#)
Defines an abstract [RestRequest](#) that can be executed by a fully configurable [RestClient](#).

Traits§

- [BuildStrategy](#)
[RestRequest](#) build strategy for the API being interacted with.
- [HttpParser](#)

Utilised by a [RestClient](#) to deserialise [RestRequest::Response](#), and upon failure parses API errors returned from the server.

[barter_integration](#)0.5.3

[BuildStrategy](#)

[Required Methods](#)

- [build](#)

[Object Safety](#)

[Implementors](#)

[In barter_integration::protocol::http](#)

[?](#)
[Change settings](#)

Trait [barter_integration::protocol::http::BuildStrategy](#)

Copy item path

[source](#) ·

[-]

```
pub trait BuildStrategy {  
  // Required method  
  fn build<Request>(  
    &self,  
    request: Request,  
    builder: RequestBuilder  
  ) -> Result<Request, SocketError>  
  where Request: RestRequest;  
}
```

Expand description

[RestRequest](#) build strategy for the API being interacted with.

An API that requires authenticated [RestRequests](#) will likely utilise the configurable [RequestSigner](#) to sign the requests before building.

An API that requires no authentication may just add mandatory request headers to the [RestRequest](#) before building.

Required Methods§

[source](#)

```
fn build<Request>( &self, request: Request, builder:
RequestBuilder ) -> Result<Request, SocketError>
where Request: RestRequest,
```

Use a [RestRequest](#) and [request::RequestBuilder](#) to construct a [request::Request](#) that is ready for executing.

It is expected that any signing or performed during this method, or the addition of any request headers.

Object Safety§

This trait is **not** [object safe](#).

Implementors§

[source§](#)

```
impl BuildStrategy for PublicNoHeaders
```

[source§](#)

```
impl<Sig, Hmac, SigEncoder> BuildStrategy for
RequestSigner<Sig, Hmac, SigEncoder>
where Sig: Signer, Hmac: Mac + Clone, SigEncoder:
Encoder,
```



[barter_integration](#)0.5.3

[HttpParser](#)

[Required Associated Types](#)

- [ApiError](#)
- [OutputError](#)

[Required Methods](#)

- [parse_api_error](#)

[Provided Methods](#)

- [parse](#)

[Object Safety](#)

[Implementors](#)

[In barter_integration::protocol::http](#)

[?](#)

[Change settings](#)

Trait [barter_integration::protocol::http::HttpParser](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub trait HttpParser {
    type ApiError: DeserializeOwned;
    type OutputError: From<SocketError>;

    // Required method
    fn parse\_api\_error(
        &self,
        status: StatusCode,
        error: Self::ApiError
    ) -> Self::OutputError;

    // Provided method
    fn parse<Response>(
        &self,
        status: StatusCode,
        payload: &[u8]
    ) -> Result<Response, Self::OutputError>
        where Response: DeserializeOwned { ... }
}
```

Expand description

Utilised by a [RestClient](#) to deserialise [RestRequest::Response](#), and upon failure parses API errors returned from the server.

Required Associated Types§

[source](#)

type [ApiError](#): [DeserializeOwned](#)

[source](#)

type [OutputError](#): [From](#)<[SocketError](#)>

Required Methods§

[source](#)

```
fn parse\_api\_error( &self, status: StatusCode, error: Self::ApiError ) -> Self::OutputError
```

If [parse](#) fails to deserialise the `Ok(Response)`, this function parses to parse the API [Self::\[ApiError\]\(#\)](#) associated with the response.

Provided Methods§

[source](#)

```
fn parse<Response>( &self, status: StatusCode, payload: &[u8] ) -> Result<Response, Self::OutputError>  
where Response: DeserializeOwned,
```

Attempt to parse a [StatusCode](#) & bytes payload into a deserialisable Response.

Object Safety§

This trait is **not** [object safe](#).

Implementors§

protocol - http - public



[barter_integration](#)0.5.3

[Module public](#)

- [Structs](#)

[In barter_integration::protocol::http](#)

[?](#)

[Change settings](#)

Module [barter_integration::protocol::http::public](#) **Copy item path**

[source](#) ·

Expand description

Defines a default [BuildStrategy](#) that builds a non-authenticated [Http RestRequest](#) with no headers.

Structs§

- [PublicNoHeaders](#)
[RestRequest BuildStrategy](#) that builds a non-authenticated [Http request](#) with no headers.

[barter_integration](#)0.5.3

[PublicNoHeaders](#)

[Trait Implementations](#)

- [BuildStrategy](#)
- [Clone](#)
- [Copy](#)
- [Debug](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)

- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In barter_integration::protocol::http::public](#)

[?](#)

[Change settings](#)

Struct [barter_integration::protocol::http::public::PublicNoHeaders](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct PublicNoHeaders;
```

Expand description

[RestRequest BuildStrategy](#) that builds a non-authenticated Http request with no headers.

Trait Implementations§

[source§](#)

impl [BuildStrategy](#) for [PublicNoHeaders](#)

[source§](#)

```
fn build<Request>( &self, _: Request, builder: RequestBuilder ) ->
Result<Request, SocketError>
```

Use a [RestRequest](#) and [request::RequestBuilder](#) to construct a [request::Request](#) that is ready for executing. [Read more](#)
[source§](#)

impl [Clone](#) for [PublicNoHeaders](#)

[source§](#)

fn [clone](#)(&self) -> [PublicNoHeaders](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source](#)§

fn [clone_from](#)(&mut self, source: [Self](#))

Performs copy-assignment from source. [Read more](#)
[source](#)§

impl [Debug](#) for [PublicNoHeaders](#)

[source](#)§

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source](#)§

impl [Copy](#) for [PublicNoHeaders](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [PublicNoHeaders](#)

§

impl [Send](#) for [PublicNoHeaders](#)

§

impl [Sync](#) for [PublicNoHeaders](#)

§

impl [Unpin](#) for [PublicNoHeaders](#)

§

impl [UnwindSafe](#) for [PublicNoHeaders](#)

Blanket Implementations§

[source](#)§

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
fn in\_current\_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

```
impl<T> Same for T
```

[§](#)

```
type Output = T
```

Should always be Self
[source§](#)

```
impl<T> ToOwned for T  
where T: Clone,
```

[§](#)

```
type Owned = T
```

The resulting type after obtaining ownership.
[source§](#)

```
fn to\_owned(&self) -> T
```

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source§](#)

```
fn clone\_into(&self, target: &mut T)
```

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

protocol - http - rest

[barter_integration](#)0.5.3

[Module rest](#)

- [Modules](#)
- [Traits](#)

[In barter_integration::protocol::http](#)

[?](#)

[Change settings](#)

Module [barter_integration::protocol::http::rest](#)

Copy item path

[source](#) ·

[-]

Expand description

Defines an abstract [RestRequest](#) that can be executed by a fully configurable [RestClient](#).

Modules§

- [client](#)
Configurable [client::RestClient](#) capable of executing signed [RestRequests](#) and parsing responses.

Traits§

- [RestRequest](#)
Http REST request that can be executed by a [RestClient](#).

barter_integration0.5.3

RestRequest

Required Associated Types

- [Body](#)
- [QueryParams](#)
- [Response](#)

Required Methods

- [method](#)
- [metric_tag](#)
- [path](#)

Provided Methods

- [body](#)
- [query_params](#)
- [timeout](#)

Object Safety

Implementors

In [barter_integration::protocol::http::rest](#)

?

[Change settings](#)

Trait [barter_integration::protocol::http::rest::RestRequest](#)

Copy item path

[source](#) ·

```
pub trait RestRequest {  
    type Response: DeserializeOwned;  
    type QueryParams: Serialize;  
    type Body: Serialize;  
  
    // Required methods  
    fn path() -> &'static str;  
    fn method() -> Method;  
    fn metric\_tag() -> Tag;
```

```
// Provided methods
fn query\_params(&self) -> Option<&Self::QueryParams> { ... }
fn body(&self) -> Option<&Self::Body> { ... }
fn timeout() -> Duration { ... }
}
```

Expand description

Http REST request that can be executed by a [RestClient](#).

Required Associated Types§

[source](#)

type [Response](#): [DeserializeOwned](#)

Expected response type if this request was successful.

[source](#)

type [QueryParams](#): [Serialize](#)

Serialisable query parameters type - use unit struct () if not required for this request.

[source](#)

type [Body](#): [Serialize](#)

Serialisable Body type - use unit struct () if not required for this request.

Required Methods§

[source](#)

fn [path](#)() -> &'static [str](#)

Additional [Url](#) path to the resource.

[source](#)

fn [method](#)() -> [Method](#)

Http [request::Method](#) of this request.

[source](#)

fn [metric_tag\(\)](#) -> [Tag](#)

[Metric Tag](#) that identifies this request.

Provided Methods§

[source](#)

fn [query_params](#)(&self) -> [Option](#)<&Self::[QueryParams](#)>

Optional query parameters for this request.

[source](#)

fn [body](#)(&self) -> [Option](#)<&Self::[Body](#)>

Optional Body for this request.

[source](#)

fn [timeout](#)() -> [Duration](#)

Http request timeout [Duration](#).

Object Safety§

This trait is **not** [object safe](#).

Implementors§

protocol - http - rest - client



[barter_integration](#)0.5.3

[RestClient](#)

[Fields](#)

- [base_url](#)
- [http_client](#)
- [metric_tx](#)
- [parser](#)
- [strategy](#)

Methods

- [build](#)
- [execute](#)
- [measured_execution](#)
- [new](#)

Trait Implementations

- [Debug](#)

Auto Trait Implementations

- [!RefUnwindSafe](#)
- [!UnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In

[barter_integration::protocol::http::rest::client](#)

[?](#)

[Change settings](#)

Struct [barter_integration::protocol::http::rest::client::RestClient](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct RestClient<'a, Strategy, Parser> {
    pub http_client: Client,
    pub base_url: &'a str,
    pub metric_tx: UnboundedSender<Metric>,
    pub strategy: Strategy,
    pub parser: Parser,
}
```

Expand description

Configurable REST client capable of executing signed [RestRequests](#). Use this when integrating APIs that require Http in order to interact with resources. Each API will require a specific combination of [Signer](#), [Mac](#), signature [Encoder](#), and [HttpParser](#).

Fields§

[\\$http_client](#): [Client](#)

HTTP [request::Client](#) for executing signed [request::Requests](#).

[\\$base_url](#): &'a [str](#)

Base Url of the API being interacted with.

[\\$metric_tx](#): [UnboundedSender](#)<[Metric](#)>

[Metric](#) transmitter for sending observed execution measurements to an external receiver.

[\\$strategy](#): Strategy

[RestRequest](#) build strategy for the API being interacted with that implements [BuildStrategy](#).

An authenticated [RestClient](#) will utilise API specific [Signer](#) logic, a hashable [Mac](#), and a signature [Encoder](#). Where as a non authorised [RestRequest](#) may add any mandatory request headers that are required.

[\\$parser](#): Parser

[HttpParser](#) that deserialises [RestRequest::Responses](#), and upon failure parses API errors returned from the server.

Implementations§

[source§](#)

```
impl<'a, Strategy, Parser> RestClient<'a, Strategy, Parser>
where Strategy: BuildStrategy, Parser: HttpParser,
```

[source](#)

```
pub async fn execute<Request>( &self, request: Request ) ->
Result<Request::Response, Parser::OutputError>
where Request: RestRequest,
```

Execute the provided [RestRequest](#).

[source](#)

```
pub fn build<Request>(&self, request: Request) -> Result<Request,
SocketError>
where Request: RestRequest,
```

Use the provided [RestRequest](#) to construct a signed Http [request::Request](#).

[source](#)

```
pub async fn measured\_execution<Request>( &self, request:
Request ) -> Result<(StatusCode, Bytes), SocketError>
where Request: RestRequest,
```

Execute the built [request::Request](#) using the [request::Client](#).

Measures the Http request round trip duration and sends the associated [Metric](#) via the [Metric](#) transmitter.

[source§](#)

```
impl<'a, Strategy, Parser> RestClient<'a, Strategy, Parser>
```

[source](#)

```
pub fn new( base_url: &'a str, metric_tx: UnboundedSender<Metric>,
strategy: Strategy, parser: Parser ) -> Self
```

Construct a new [Self](#) using the provided configuration.

Trait Implementations§

[source§](#)

```
impl<'a, Strategy: Debug, Parser: Debug> Debug for
RestClient<'a, Strategy, Parser>
```

[source§](#)

```
fn fmt(&self, f: &mut Formatter<'_>) -> Result
```

Formats the value using the given formatter. [Read more](#)

Auto Trait Implementations§

§

```
impl<'a, Strategy, Parser> !RefUnwindSafe for  
RestClient<'a, Strategy, Parser>
```

§

```
impl<'a, Strategy, Parser> Send for RestClient<'a,  
Strategy, Parser>  
where Parser: Send, Strategy: Send,
```

§

```
impl<'a, Strategy, Parser> Sync for RestClient<'a, Strategy,  
Parser>  
where Parser: Sync, Strategy: Sync,
```

§

```
impl<'a, Strategy, Parser> Unpin for RestClient<'a,  
Strategy, Parser>  
where Parser: Unpin, Strategy: Unpin,
```

§

```
impl<'a, Strategy, Parser> !UnwindSafe for RestClient<'a,  
Strategy, Parser>
```

Blanket Implementations§

[source](#)§

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)§

```
fn type\_id(&self) -> TypeId
```

Gets the TypeId of self. [Read more](#)
[source](#)§

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
impl<T, U> Into<U> for T
where U: From<T>,
```

[source](#)§

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of `From<T>` for `U` chooses to do.

[source](#)§

```
impl<T> Same for T
```

§

```
type Output = T
```

Should always be `Self`

[source](#)§

```
impl<T, U> TryFrom<U> for T
where U: Into<T>,
```

§

```
type Error = Infallible
```

The type returned in the event of a conversion error.

[source](#)§

```
fn try_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source](#)§

```
impl<T, U> TryInto<U> for T
where U: TryFrom<T>,
```

§

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source](#)§

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.
[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)



[barter_integration](#)0.5.3

[Module client](#)

- [Structs](#)

[In barter_integration::protocol::http::rest](#)



[?](#)

[Change settings](#)

Module [barter_integration::protocol::http::rest::client](#)

Copy item path

[source](#) ·

[−]

Expand description

Configurable [client::RestClient](#) capable of executing signed [RestRequests](#) and parsing responses.

Structs§

- [RestClient](#)

Configurable REST client capable of executing signed [RestRequests](#). Use this when integrating APIs that require Http in order to interact with resources. Each API will require a specific combination of [Signer](#), [Mac](#), signature [Encoder](#), and [HttpParser](#).

protocol - http - private

[barter_integration](#)0.5.3

[Module private](#)

- [Modules](#)
- [Structs](#)
- [Traits](#)

[In barter_integration::protocol::http](#)

?

[Change settings](#)

Module [barter_integration::protocol::http::private](#)

Copy item path

[source](#) ·

[−]

Expand description

Defines a configurable [RequestSigner](#) that signs Http [RestRequest](#) using API specific logic.

Modules§

- [encoder](#)
Implementations for encoding signatures generated by a [RequestSigner](#).

Structs§

- [RequestSigner](#)
Generically signs Http [RestRequests](#) utilising API specific [Signer](#) logic, a hashable [Mac](#), and a signature [Encoder](#).

Traits§

- [Signer](#)
API specific signing logic used by a [RequestSigner](#).



[barter_integration](#)0.5.3

[RequestSigner](#)

[Methods](#)

- [new](#)

[Trait Implementations](#)

- [BuildStrategy](#)
- [Clone](#)
- [Copy](#)
- [Debug](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)

- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In barter_integration::protocol::http::private](#)

[?](#)

[Change settings](#)

Struct [barter_integration::protocol::http::private::RequestSigner](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct RequestSigner<Sig, Hmac, SigEncoder> { /* private fields */ }
```

Expand description

Generically signs Http [RestRequests](#) utilising API specific [Signer](#) logic, a hashable [Mac](#), and a signature [Encoder](#).

Implementations§

[source§](#)

```
impl<Sig, Hmac, SigEncoder> RequestSigner<Sig, Hmac, SigEncoder>
```

[source](#)

```
pub fn new(signer: Sig, mac: Hmac, encoder: SigEncoder) -> Self
```

Construct a new [Self](#) using the provided API specific configuration.

Trait Implementations§

[source§](#)

impl<Sig, Hmac, SigEncoder> [BuildStrategy](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Sig: [Signer](#), Hmac: [Mac](#) + [Clone](#), SigEncoder: [Encoder](#),

[source§](#)

fn [build](#)<Request>(&self, request: Request, builder: [RequestBuilder](#)) -> [Result](#)<[Request](#), [SocketError](#)>
where Request: [RestRequest](#),

Use a [RestRequest](#) and [request::RequestBuilder](#) to construct a [request::Request](#) that is ready for executing. [Read more](#)
[source§](#)

impl<Sig: [Clone](#), Hmac: [Clone](#), SigEncoder: [Clone](#)> [Clone](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>

[source§](#)

fn [clone](#)(&self) -> [RequestSigner](#)<Sig, Hmac, SigEncoder>

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl<Sig: [Debug](#), Hmac: [Debug](#), SigEncoder: [Debug](#)> [Debug](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<Sig: [Copy](#), Hmac: [Copy](#), SigEncoder: [Copy](#)> [Copy](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>

Auto Trait Implementations§

§

impl<Sig, Hmac, SigEncoder> [RefUnwindSafe](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Hmac: [RefUnwindSafe](#), Sig: [RefUnwindSafe](#), SigEncoder: [RefUnwindSafe](#),

[§](#)

impl<Sig, Hmac, SigEncoder> [Send](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Hmac: [Send](#), Sig: [Send](#), SigEncoder: [Send](#),

[§](#)

impl<Sig, Hmac, SigEncoder> [Sync](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Hmac: [Sync](#), Sig: [Sync](#), SigEncoder: [Sync](#),

[§](#)

impl<Sig, Hmac, SigEncoder> [Unpin](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Hmac: [Unpin](#), Sig: [Unpin](#), SigEncoder: [Unpin](#),

[§](#)

impl<Sig, Hmac, SigEncoder> [UnwindSafe](#) for [RequestSigner](#)<Sig, Hmac, SigEncoder>
where Hmac: [UnwindSafe](#), Sig: [UnwindSafe](#), SigEncoder: [UnwindSafe](#),

Blanket Implementations[§](#)

[source](#)[§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)[§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source](#)[§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source](#)§

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of `From<T>` for `U` chooses to do.

[source](#)§

impl<T> [Same](#) for T

§

type [Output](#) = T

Should always be `Self`

[source](#)§

impl<T> [ToOwned](#) for T
where T: [Clone](#),

§

type [Owned](#) = T

The resulting type after obtaining ownership.

[source](#)§

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)§

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)§

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

§

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source](#)

fn [vzip](#)(self) -> V

[source](#)

impl<T> [WithSubscriber](#) for T

[source](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)



barter_integration0.5.3

Signer

Required Associated Types

- [Config](#)

Required Methods

- [build_signed_request](#)
- [bytes_to_sign](#)
- [config](#)

Object Safety

Implementors

In [barter_integration::protocol::http::private](#)



[?](#)

[Change settings](#)

Trait [barter_integration::protocol::http::private::Signer](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub trait Signer {
  type Config<'a>
  where Self: 'a;

  // Required methods
  fn config<'a, Request>(
    &'a self,
    request: Request,
    builder: &RequestBuilder
  ) -> Result<Self::Config<'a>, SocketError>
  where Request: RestRequest;
  fn bytes\_to\_sign<'a>(config: &Self::Config<'a>) -> Bytes;
  fn build\_signed\_request<'a>(
    config: Self::Config<'a>,
    builder: RequestBuilder,
    signature: String
```

```
    ) -> Result<Request, SocketError>;  
}
```

Expand description

API specific signing logic used by a [RequestSigner](#).

Required Associated Types§

[source](#)

type [Config](#)<'a> where Self: 'a

Configuration required to sign the [RestRequest](#)s for this API server.

Required Methods§

[source](#)

fn [config](#)<'a, [Request](#)>(&'a self, request: [Request](#), builder:
&[RequestBuilder](#)) -> [Result](#)<Self::[Config](#)<'a>, [SocketError](#)>
where [Request](#): [RestRequest](#),

Generates a [Self::\[Config\]\(#\)](#) for this [RestRequest](#) and [RequestBuilder](#).

§Examples

[§Private REST Request: FTX](#)

—

```
fn config<Request>(&self, _: Request, _: &RequestBuilder) -> Self::Config  
where  
    Request: RestRequest  
{  
    FtxSignConfig {  
        api_key: self.api_key.as_str(),  
        time: Utc::now(),  
        method: Request::method(),  
        path: Request::path()  
    }  
}
```

[source](#)

fn [bytes_to_sign](#)<'a>(config: &Self::[Config](#)<'a>) -> [Bytes](#)

Generate the bytes to sign from the provided [Self::\[Config\]\(#\)](#).

§Examples

§Private REST Request: FTX

—

```
fn bytes_to_sign(config: &Self::Config) -> Result<&[u8], SocketError> {  
    Ok(format!("{}", config.time, config.method, config.path).as_bytes())  
}
```

[source](#)

fn **build_signed_request**<'a>(config: Self::Config<'a>, builder: RequestBuilder, signature: String) -> Result<Request, SocketError>

Build a signed [request::Request](#) from the provided [Self::Config](#), [RequestBuilder](#), and generated cryptographic signature String.

§Examples

§Private REST Request: FTX

—

```
fn build_signed_request(config: Self::Config, builder: RequestBuilder, signature: String) -> Result<Request, SocketError> {  
    // Add Ftx required Headers & build request::Request  
    builder  
        .header(HEADER_FTX_KEY, &self.api_key)  
        .header(HEADER_FTX_SIGN, &signature)  
        .header(HEADER_FTX_TS, &time)  
        .build()  
        .map_err(SocketError::from)  
}
```

Object Safety§

This trait is **not** [object safe](#).

Implementors§

protocol - http - private - encoder



barter_integration0.5.3

Module encoder

- [Structs](#)
- [Traits](#)

In barter_integration::protocol::http::private

[?](#)
[Change settings](#)

Module [barter_integration::protocol::http::private::encoder](#)

Copy item path

[source](#) · [\[-\]](#)
Expand description

Implementations for encoding signatures generated by a [RequestSigner](#).

Structs§

- [HexEncoder](#)
Encodes bytes data as a hex String using lowercase characters.

Traits§

- [Encoder](#)
Encodes bytes data.

barter_integration0.5.3

Encoder

Required Methods

- [encode](#)

[Object Safety](#)

[Implementors](#)

[In](#)

[barter_integration::protocol::http::private::encoder](#)

[?](#)

[Change settings](#)

Trait [barter_integration::protocol::http::private::encoder::Encoder](#)

Copy item path

[source](#) ·

[−]

```
pub trait Encoder {  
    // Required method  
    fn encode<Bytes>(&self, data: Bytes) -> String  
    where Bytes: AsRef<[u8]>;  
}
```

Expand description

Encodes bytes data.

Required Methods§

[source](#)

```
fn encode<Bytes>(&self, data: Bytes) -> String  
where Bytes: AsRef<[u8]>,
```

Encodes the bytes data into some String format.

Object Safety§

This trait is **not** [object safe](#).

Implementors§

[source§](#)

impl [Encoder](#) for [HexEncoder](#)



barter_integration0.5.3

HexEncoder

Trait Implementations

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Encoder](#)

Auto Trait Implementations

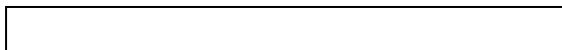
- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In

barter_integration::protocol::http::private::encoder



[?](#)
[Change settings](#)

Struct [barter_integration::protocol::http::private::encoder::HexEncoder](#)

Copy item path

[source](#) · [\[-\]](#)

```
pub struct HexEncoder;
```

Expand description

Encodes bytes data as a hex String using lowercase characters.

Trait Implementations§

[source§](#)

impl [Clone](#) for [HexEncoder](#)

[source§](#)

```
fn clone(&self) -> HexEncoder
```

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

```
fn clone\_from(&mut self, source: Self)
```

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [HexEncoder](#)

[source§](#)

```
fn fmt(&self, f: &mut Formatter<'_,>) -> Result
```

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [Encoder](#) for [HexEncoder](#)

[source§](#)

```
fn encode<Bytes>(&self, data: Bytes) -> String  
where Bytes: AsRef<[u8]>,
```

Encodes the bytes data into some String format.

[source§](#)

impl [Copy](#) for [HexEncoder](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [HexEncoder](#)

§

impl [Send](#) for [HexEncoder](#)

§

impl [Sync](#) for [HexEncoder](#)

§

impl [Unpin](#) for [HexEncoder](#)

§

impl [UnwindSafe](#) for [HexEncoder](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)

[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)

[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)

[source§](#)

**impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,**

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.

[source§](#)

```
impl<T, U> TryInto<U> for T
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.
[source§](#)

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.
[source§](#)

```
impl<V, T> VZip<V> for T
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

protocol - websocket



[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Function [barter_integration::protocol::websocket::connect](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub async fn connect<R>(request: R) -> Result<WebSocket, SocketError>
where
```

```
    R: IntoClientRequest + Unpin + Debug,
```

Expand description

Connect asynchronously to a [WebSocket](#) server.

[barter_integration](#)0.5.3

[WebSocket](#)

[Aliased type](#)

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Type Alias [barter_integration::protocol::websocket::WebSocket](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub type WebSocket = WebSocketStream<MaybeTlsStream<TcpStream>>;
```

Expand description

Convenient type alias for a tungstenite WebSocketStream.

Aliased Type§

```
struct WebSocket { /* private fields */ }
```

[barter_integration](#)0.5.3

[Module websocket](#)

- [Structs](#)
- [Functions](#)
- [Type Aliases](#)

[In barter_integration::protocol](#)

[?](#)

[Change settings](#)

Module [barter_integration::protocol::websocket](#)

Copy item path

[source](#) · [\[-\]](#)

Expand description

Contains useful WebSocket type aliases and a default WebSocket implementation of a [StreamParser](#).

Structs§

- [WebSocketParser](#)
Default [StreamParser](#) implementation for a [WebSocket](#).

Functions§

- [connect](#)
Connect asynchronously to a [WebSocket](#) server.
- [is_websocket_disconnected](#)
Determine whether a [WsError](#) indicates the [WebSocket](#) has disconnected.
- [process_binary](#)

Process a payload of Vec<u8> bytes by deserialising into an ExchangeMessage.

- [process_close_frame](#)
Basic process for a [WebSocket](#) CloseFrame message. Logs the payload at trace level.
- [process_frame](#)
Basic process for a [WebSocket](#) Frame message. Logs the payload at trace level.
- [process_ping](#)
Basic process for a [WebSocket](#) ping message. Logs the payload at trace level.
- [process_pong](#)
Basic process for a [WebSocket](#) pong message. Logs the payload at trace level.
- [process_text](#)
Process a payload of String by deserialising into an ExchangeMessage.

Type Aliases§

- [WebSocket](#)
Convenient type alias for a tungstenite WebSocketStream.
- [WsError](#)
Communicative type alias for a tungstenite [WebSocket](#) Error.
- [WsMessage](#)
Communicative type alias for a tungstenite [WebSocket](#) Message.
- [WsSink](#)
Convenient type alias for the Sink half of a tungstenite [WebSocket](#).
- [WsStream](#)
Convenient type alias for the Stream half of a tungstenite [WebSocket](#).

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Function [barter_integration::protocol::websocket::is_websocket_disconnected](#)

Copy item path

[source](#) ·

[−]

```
pub fn is_websocket_disconnected(error: &WsError) -> bool
```

Expand description

Determine whether a [WsError](#) indicates the [WebSocket](#) has disconnected.

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Function [barter_integration::protocol::websocket::process_ping](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub fn process_ping<ExchangeMessage>(
    ping: Vec<u8>
) -> Option<Result<ExchangeMessage, SocketError>>
```

Expand description

Basic process for a [WebSocket](#) ping message. Logs the payload at trace level.

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Function [barter_integration::protocol::websocket::process_frame](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub fn process_frame<ExchangeMessage>(
  frame: Frame
) -> Option<Result<ExchangeMessage, SocketError>>
```

Expand description

Basic process for a [WebSocket](#) Frame message. Logs the payload at trace level.

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)
[Change settings](#)

Function [barter_integration::protocol::websocket::process_close_frame](#)

Copy item path

[source](#) · [\[-\]](#)

```
pub fn process_close_frame<ExchangeMessage>(
  close_frame: Option<CloseFrame<'_>>
) -> Option<Result<ExchangeMessage, SocketError>>
```

Expand description

Basic process for a [WebSocket](#) CloseFrame message. Logs the payload at trace level.

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)
[Change settings](#)

Function [barter_integration::protocol::websocket::process_text](#)

Copy item path

[source](#) ·

[–]

```
pub fn process_text<ExchangeMessage>(
    payload: String
) -> Option<Result<ExchangeMessage, SocketError>>
where
    ExchangeMessage: DeserializeOwned,
```

Expand description

Process a payload of String by deserialising into an ExchangeMessage.

[barter_integration](#)0.5.3

[WsStream](#)

[Aliased type](#)

In [barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Type Alias [barter_integration::protocol::websocket::WsStream](#)

Copy item path

[source](#) ·

[–]

```
pub type WsStream = SplitStream<WebSocket>;
```

Expand description

Convenient type alias for the Stream half of a tungstenite [WebSocket](#).

Aliased Type§

```
struct WsStream(/* private fields */);
```

barter_integration0.5.3

In barter_integration::protocol::websocket

[?](#)
[Change settings](#)

Function barter_integration::protocol::websocket::process_pong

Copy item path

[source](#) ·

[-]

```
pub fn process_pong<ExchangeMessage>(
    pong: Vec<u8>
) -> Option<Result<ExchangeMessage, SocketError>>
```

Expand description

Basic process for a [WebSocket](#) pong message. Logs the payload at trace level.

barter_integration0.5.3

WsSink

[Aliased type](#)

In barter_integration::protocol::websocket

[?](#)
[Change settings](#)

Type Alias [barter_integration::protocol::websocket::WsSink](#)

Copy item path

[source](#) ·

[−]

```
pub type WsSink = SplitSink<WebSocket, WsMessage>;
```

Expand description

Convenient type alias for the Sink half of a tungstenite [WebSocket](#).

Aliased Type§

```
struct WsSink { /* private fields */ }
```

[barter_integration](#)0.5.3

[WsError](#)

[Aliased type](#)

[Variants](#)

- [AlreadyClosed](#)
- [Capacity](#)
- [ConnectionClosed](#)
- [Http](#)
- [HttpFormat](#)
- [Io](#)
- [Protocol](#)
- [SendQueueFull](#)
- [Tls](#)
- [Url](#)
- [Utf8](#)

[In barter_integration::protocol::websocket](#)

?

[Change settings](#)

Type Alias [barter_integration::protocol::websocket::WsError](#)

Copy item path

[source](#) ·

[−]

```
pub type WsError = Error;
```

Expand description

Communicative type alias for a tungstenite [WebSocket](#) Error.

Aliased Type§

```
enum WsError {  
    ConnectionClosed,  
    AlreadyClosed,  
    Io(Error),  
    Tls(TlsError),  
    Capacity(CapacityError),  
    Protocol(ProtocolError),  
    SendQueueFull(Message),  
    Utf8,  
    Url(UrlError),  
    Http(Response<Option<Vec<u8>>>),  
    HttpFormat(Error),  
}
```

Variants§

§

ConnectionClosed

WebSocket connection closed normally. This informs you of the close. It's not an error as such and nothing wrong happened.

This is returned as soon as the close handshake is finished (we have both sent and received a close frame) on the server end and as soon as the server has closed the underlying connection if this endpoint is a client.

Thus when you receive this, it is safe to drop the underlying connection.

Receiving this error means that the WebSocket object is not usable anymore and the only meaningful action with it is dropping it.

§

AlreadyClosed

Trying to work with already closed connection.

Trying to read or write after receiving `ConnectionClosed` causes this.

As opposed to `ConnectionClosed`, this indicates your code tries to operate on the connection when it really shouldn't anymore, so this really indicates a programmer error on your part.

§

Io([Error](#))

Input-output error. Apart from `WouldBlock`, these are generally errors with the underlying connection and you should probably consider them fatal.

§

Tls([TlsError](#))

TLS error.

Note that this error variant is enabled unconditionally even if no TLS feature is enabled, to provide a feature-agnostic API surface.

§

Capacity([CapacityError](#))

- When reading: buffer capacity exhausted.
- When writing: your message is bigger than the configured max message size (64MB by default).

§

Protocol([ProtocolError](#))

Protocol violation.

§

SendQueueFull([Message](#))

Message send queue full.

§

Utf8

UTF coding error.

§

Url([UrlError](#))

Invalid URL.

§

Http([Response](#) < [Option](#) < [Vec](#) < [u8](#) > > >)

HTTP error.

§

HttpFormat([Error](#))

HTTP format error.



[barter_integration](#)0.5.3

[WebSocketParser](#)

[Trait Implementations](#)

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize](#) < 'de' >
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StreamParser](#)
- [StructuralPartialEq](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In barter_integration::protocol::websocket

?

[Change settings](#)

Struct [barter_integration::protocol::websocket::WebSocketParser](#)

Copy item path

[source](#) ·

[–]

```
pub struct WebSocketParser;
```

Expand description

Default [StreamParser](#) implementation for a [WebSocket](#).

Trait Implementations§

[source§](#)

impl [Clone](#) for [WebSocketParser](#)

[source§](#)

fn [clone](#)(&self) -> [WebSocketParser](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [WebSocketParser](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [WebSocketParser](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::[Error](#)>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Hash](#) for [WebSocketParser](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [WebSocketParser](#)

[source§](#)

fn [cmp](#)(&self, other: [&WebSocketParser](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

```
fn max(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

```
fn min(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source§](#)

```
impl PartialEq for WebSocketParser
```

[source§](#)

```
fn eq(&self, other: &WebSocketParser) -> bool
```

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

```
impl PartialOrd for WebSocketParser
```

[source§](#)

```
fn partial_cmp(&self, other: &WebSocketParser) ->  
Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source](#)

impl [Serialize](#) for [WebSocketParser](#)

[source](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)

impl [StreamParser](#) for [WebSocketParser](#)

[§](#)

type [Stream](#) = [WebSocketStream](#)<[MaybeTlsStream](#)<[TcpStream](#)>>

[§](#)

type [Message](#) = [Message](#)

[§](#)

type [Error](#) = [Error](#)

[source](#)

```
fn parse<Output>( input: Result<Self::Message, Self::Error> ) ->  
Option<Result<Output, SocketError>>  
where Output: DeserializeOwned,
```

[source§](#)

```
impl Copy for WebSocketParser
```

[source§](#)

```
impl Eq for WebSocketParser
```

[source§](#)

```
impl StructuralPartialEq for WebSocketParser
```

Auto Trait Implementations§

§

```
impl RefUnwindSafe for WebSocketParser
```

§

```
impl Send for WebSocketParser
```

§

```
impl Sync for WebSocketParser
```

§

```
impl Unpin for WebSocketParser
```

§

```
impl UnwindSafe for WebSocketParser
```

Blanket Implementations§

[source§](#)

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self
[source](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
impl<T> DeserializeOwned for T  
where T: for<'de> Deserialize<'de>,
```



[barter_integration](#)0.5.3

[WsMessage](#)

[Aliased type](#)

[Variants](#)

- [Binary](#)
- [Close](#)
- [Frame](#)
- [Ping](#)

- [Pong](#)
- [Text](#)

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Type Alias [barter_integration::protocol::websocket::WsMessage](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub type WsMessage = Message;
```

Expand description

Communicative type alias for a tungstenite [WebSocket](#) Message.

Aliased Type§

```
enum WsMessage {
    Text(String),
    Binary(Vec<u8>),
    Ping(Vec<u8>),
    Pong(Vec<u8>),
    Close(Option<CloseFrame<'static>>),
    Frame(Frame),
}
```

Variants§

§

Text([String](#))

A text WebSocket message

§

Binary([Vec](#)<[u8](#)>)

A binary WebSocket message

§

Ping([Vec](#)<[u8](#)>)

A ping message with the specified payload

The payload here must have a length less than 125 bytes

[§](#)

Pong([Vec](#)<[u8](#)>)

A pong message with the specified payload

The payload here must have a length less than 125 bytes

[§](#)

Close([Option](#)<[CloseFrame](#)<'static'>>)

A close message with the optional close frame.

[§](#)

Frame([Frame](#))

Raw frame. Note, that you're not going to get this value while reading the message.

[barter_integration](#)0.5.3

[In barter_integration::protocol::websocket](#)

[?](#)

[Change settings](#)

Function [barter_integration::protocol::websocket::process_binary](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub fn process_binary<ExchangeMessage>(
    payload: Vec<u8>
) -> Option<Result<ExchangeMessage, SocketError>>
where
    ExchangeMessage: DeserializeOwned,
```

Expand description

Process a payload of Vec<u8> bytes by deserialising into an ExchangeMessage.

model



barter_integration0.5.3

Side

Variants

- [Buy](#)
- [Sell](#)

Trait Implementations

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)

- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In `barter\_integration::model`](#)

[?](#)

[Change settings](#)

Enum [barter_integration::model::Side](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub enum Side {
    Buy,
    Sell,
}
```

Expand description

[Side](#) of a trade or position - Buy or Sell.

Variants§

§

Buy

§

Sell

Trait Implementations§

[source](#)§

impl [Clone](#) for [Side](#)

[source§](#)

fn [clone](#)(&self) -> [Side](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)

[source§](#)

impl [Debug](#) for [Side](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<'de> [Deserialize](#)<'de> for [Side](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::[Error](#)>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)

[source§](#)

impl [Display](#) for [Side](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl [Hash](#) for [Side](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)

1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: &[\[Self\]](#), state: &mut [H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [Side](#)

[source§](#)

fn [cmp](#)(&self, other: &[Side](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [Side](#)

[source§](#)

fn [eq](#)(&self, other: &[Side](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [Side](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[Side](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)

1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)

1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)

1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)

1.0.0 · [source§](#)

fn [ge](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)

[source§](#)

impl [Serialize](#) for [Side](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::[Ok](#), __S::[Error](#)>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)

[source§](#)

impl [Copy](#) for [Side](#)

[source§](#)

impl [Eq](#) for [Side](#)

[source](#)§

impl [StructuralPartialEq](#) for [Side](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Side](#)

§

impl [Send](#) for [Side](#)

§

impl [Sync](#) for [Side](#)

§

impl [Unpin](#) for [Side](#)

§

impl [UnwindSafe](#) for [Side](#)

Blanket Implementations§

[source](#)§

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)§

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source](#)§

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source](#)§

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: [?Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + [?Sized](#), K: [Borrow](#)<Q> + [?Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering. [source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + [?Sized](#), K: [Borrow](#)<Q> + [?Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + [?Sized](#), K: [Borrow](#)<Q> + [?Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal. [source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

**impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,**

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
impl<T> DeserializeOwned for T  
where T: for<'de> Deserialize<'de>,
```

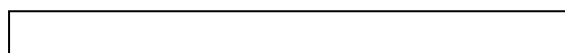


[barter_integration](#)0.5.3

[Module model](#)

- [Modules](#)
- [Structs](#)
- [Enums](#)

[In crate barter_integration](#)



[?](#)
[Change settings](#)

Module [barter_integration::model](#)

Copy item path

[source](#) ·

Expand description

Foundational data structures that define the building blocks used by the rest of the Barter ecosystem.

eg/ Market, Exchange, Instrument, Symbol, etc.

Modules§

- [instrument](#)
[Instrument](#) related data structures.

Structs§

- [Exchange](#)
Barter representation of an [Exchange](#)'s name.
- [Market](#)
Represents a unique combination of an [Exchange](#) & an [Instrument](#).
- [MarketId](#)
Barter new type representing a unique String identifier for a [Market](#), where a [Market](#) represents an [Instrument](#) being traded on an [Exchange](#).
- [SubscriptionId](#)
New type representing a unique String identifier for a stream that has been subscribed to. This is used to identify data structures received over the socket.

Enums§

- [Side](#)
[Side](#) of a trade or position - Buy or Sell.

barter_integration0.5.3

SubscriptionId

Tuple Fields

- [0](#)

Trait Implementations

- [AsRef<str>](#)
- [Clone](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [From<S>](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)

- [WithSubscriber](#)

In barter_integration::model

?

[Change settings](#)

Struct [barter_integration::model::](#)

[SubscriptionId](#)

Copy item path

[source](#) ·

[−]

```
pub struct SubscriptionId(pub String);
```

Expand description

New type representing a unique `String` identifier for a stream that has been subscribed to. This is used to identify data structures received over the socket.

For example, Barter-Data uses this identifier to associate received data structures from the exchange with the original Barter-Data Subscription that was actioned over the socket.

Note: Each exchange will require the use of different `String` identifiers depending on the data structures they send.

eg/ [SubscriptionId](#) of an `FtxTrade` is “{BASE}/{QUOTE}” (ie/ market). eg/ [SubscriptionId](#) of a `BinanceTrade` is “{base}{symbol}@trade” (ie/ channel).

Tuple Fields§

§0: [String](#)

Trait Implementations§

[source§](#)

```
impl AsRef<str> for SubscriptionId
```

[source§](#)

```
fn as\_ref(&self) -> &str
```

Converts this type into a shared reference of the (usually inferred) input type.

[source§](#)

impl [Clone](#) for [SubscriptionId](#)

[source§](#)

fn [clone](#)(&self) -> [SubscriptionId](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [SubscriptionId](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [SubscriptionId](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::[Error](#)>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [SubscriptionId](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<S> [From](#)<S> for [SubscriptionId](#)
where S: [Into](#)<[String](#)>,

[source§](#)

fn [from](#)(input: S) -> Self

Converts to this type from the input type.
[source§](#)

impl [Hash](#) for [SubscriptionId](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

**fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),**

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [SubscriptionId](#)

[source§](#)

fn [cmp](#)(&self, other: [&SubscriptionId](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

**fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

**fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

**fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),**

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [SubscriptionId](#)

[source§](#)

fn [eq](#)(&self, other: &[SubscriptionId](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [SubscriptionId](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[SubscriptionId](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [SubscriptionId](#)

[source](#)§

```
fn serialize<__S>(&self, __serializer: __S) -> Result<__S::Ok,  
__S::Error>  
where __S: Serializer,
```

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)§

impl [Eq](#) for [SubscriptionId](#)

[source](#)§

impl [StructuralPartialEq](#) for [SubscriptionId](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [SubscriptionId](#)

§

impl [Send](#) for [SubscriptionId](#)

§

impl [Sync](#) for [SubscriptionId](#)

§

impl [Unpin](#) for [SubscriptionId](#)

§

impl [UnwindSafe](#) for [SubscriptionId](#)

Blanket Implementations§

[source](#)§

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)§

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

```
fn equivalent(&self, key: &K) -> bool
```

Compare self to key and return true if they are equal.
[source§](#)

```
impl<T> From<T> for T
```

[source§](#)

```
fn from(t: T) -> T
```

Returns the argument unchanged.

[source§](#)

```
impl<T> Instrument for T
```

[source§](#)

```
fn instrument(self, span: Span) -> Instrumented<Self>
```

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
fn in_current_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

```
impl<T> Same for T
```

[§](#)

type [Output](#) = T

Should always be Self
[source](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

```
impl<T, U> TryInto<U> for T
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.
[source§](#)

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.
[source§](#)

```
impl<V, T> VZip<V> for T
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
impl<T> DeserializeOwned for T
where T: for<'de> Deserialize<'de>,
```



barter_integration0.5.3

MarketId

Tuple Fields

- [0](#)

Methods

- [new](#)

Trait Implementations

- [Clone](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [From<M>](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)

- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In barter_integration::model](#)

?

[Change settings](#)

Struct [barter_integration::model::MarketId](#)

Copy item path

[source](#) ·

[−]

```
pub struct MarketId(pub String);
```

Expand description

Barter new type representing a unique String identifier for a [Market](#), where a [Market](#) represents an [Instrument](#) being traded on an [Exchange](#).

eg/ binance_btc_usdt_spot eg/ ftx_btc_usdt_future_perpetual

Tuple Fields§

§0: [String](#)

Implementations§

[source§](#)

impl [MarketId](#)

[source](#)

```
pub fn new(exchange: &Exchange, instrument: &Instrument) -> Self
```

Construct a unique String [MarketId](#) identifier for a [Market](#), where a [Market](#) represents an [Instrument](#) being traded on an [Exchange](#).

Trait Implementations§

[source§](#)

impl [Clone](#) for [MarketId](#)

[source§](#)

fn [clone](#)(&self) -> [MarketId](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [Self](#))

Performs copy-assignment from source. [Read more](#)

[source§](#)

impl [Debug](#) for [MarketId](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<'de> [Deserialize](#)<'de> for [MarketId](#)

[source§](#)

fn [deserialize](#)<D: [Deserializer](#)<'de>>(deserializer: D) -> [Result](#)<Self, D::Error>

Deserialize this value from the given Serde deserializer. [Read more](#)

[source§](#)

impl [Display](#) for [MarketId](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<'a, M> [From](#)<M> for [MarketId](#)

where M: [Into](#)<&'a [Market](#)>,

[source§](#)

fn [from](#)(market: M) -> Self

Converts to this type from the input type.
[source§](#)

impl [Hash](#) for [MarketId](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

**fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),**

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [MarketId](#)

[source§](#)

fn [cmp](#)(&self, other: [&MarketId](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

**fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

**fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

**fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),**

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [MarketId](#)

[source§](#)

fn [eq](#)(&self, other: &[MarketId](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [MarketId](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[MarketId](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [MarketId](#)

[source](#)§

```
fn serialize<__S>(&self, __serializer: __S) -> Result<__S::Ok,  
__S::Error>  
where __S: Serializer,
```

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)§

impl [Eq](#) for [MarketId](#)

[source](#)§

impl [StructuralPartialEq](#) for [MarketId](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [MarketId](#)

§

impl [Send](#) for [MarketId](#)

§

impl [Sync](#) for [MarketId](#)

§

impl [Unpin](#) for [MarketId](#)

§

impl [UnwindSafe](#) for [MarketId](#)

Blanket Implementations§

[source](#)§

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)§

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self
[source](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

```
impl<T, U> TryInto<U> for T
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.
[source§](#)

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.
[source§](#)

```
impl<V, T> VZip<V> for T
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
impl<T> DeserializeOwned for T
where T: for<'de> Deserialize<'de>,
```



barter_integration0.5.3

Market

Fields

- [exchange](#)
- [instrument](#)

Methods

- [new](#)

Trait Implementations

- [Clone](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Eq](#)
- [From<\(E, I\)>](#)
- [From<\(E, S, S, InstrumentKind\)>](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)

- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In barter_integration::model](#)

?

[Change settings](#)

Struct [barter_integration::model::Market](#)

Copy item path

[source](#) ·

[−]

```
pub struct Market {
  pub exchange: Exchange,
  pub instrument: Instrument,
}
```

Expand description

Represents a unique combination of an [Exchange](#) & an [Instrument](#).

Fields§

[\\$exchange](#): [Exchange](#)[\\$instrument](#): [Instrument](#)

Implementations§

[source](#)§

impl [Market](#)

[source](#)

```
pub fn new<E, I>(exchange: E, instrument: I) -> Self
where E: Into<Exchange>, I: Into<Instrument>,
```

Constructs a new [Market](#) using the provided [Exchange](#) & [Instrument](#).

Trait Implementations§

[source](#)§

impl [Clone](#) for [Market](#)

[source§](#)

fn [clone](#)(&self) -> [Market](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [Self](#))

Performs copy-assignment from source. [Read more](#)

[source§](#)

impl [Debug](#) for [Market](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<'de> [Deserialize](#)<'de> for [Market](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)

[source§](#)

impl<E, I> [From](#)<(E, I)> for [Market](#)
where E: [Into](#)<[Exchange](#)>, I: [Into](#)<[Instrument](#)>,

[source§](#)

fn [from](#)((exchange, instrument): (E, I)) -> Self

Converts to this type from the input type.

[source§](#)

impl<E, S> [From](#)<(E, S, S, [InstrumentKind](#))> for [Market](#)
where E: [Into](#)<[Exchange](#)>, S: [Into](#)<[Symbol](#)>,

[source§](#)

fn [from](#)((exchange, base, quote, instrument_kind): (E, S, S, [InstrumentKind](#))) -> Self

Converts to this type from the input type.
[source§](#)

impl [Hash](#) for [Market](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

**fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),**

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [Market](#)

[source§](#)

fn [cmp](#)(&self, other: [&Market](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

**fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

**fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),**

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

**fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),**

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [Market](#)

[source§](#)

fn [eq](#)(&self, other: &[Market](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [Market](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[Market](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Market](#)

[source](#)§

```
fn serialize<__S>(&self, __serializer: __S) -> Result<__S::Ok,  
__S::Error>  
where __S: Serializer,
```

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)§

impl [Eq](#) for [Market](#)

[source](#)§

impl [StructuralPartialEq](#) for [Market](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Market](#)

§

impl [Send](#) for [Market](#)

§

impl [Sync](#) for [Market](#)

§

impl [Unpin](#) for [Market](#)

§

impl [UnwindSafe](#) for [Market](#)

Blanket Implementations§

[source](#)§

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)§

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

```
fn equivalent(&self, key: &K) -> bool
```

Compare self to key and return true if they are equal.
[source§](#)

```
impl<T> From<T> for T
```

[source§](#)

```
fn from(t: T) -> T
```

Returns the argument unchanged.

[source§](#)

```
impl<T> Instrument for T
```

[source§](#)

```
fn instrument(self, span: Span) -> Instrumented<Self>
```

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
fn in_current_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

```
impl<T> Same for T
```

[§](#)

type [Output](#) = T

Should always be Self
[source](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source](#)

```
fn vzip(self) -> V
```

[source](#)

```
impl<T> WithSubscriber for T
```

[source](#)

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source](#)

```
impl<T> DeserializeOwned for T  
where T: for<'de> Deserialize<'de>,
```



[barter_integration](#)0.5.3

[Exchange](#)

[Trait Implementations](#)

- [Clone](#)
- [Debug](#)
- [Deserialize](#)<'de>
- [Display](#)
- [Eq](#)
- [From](#)<E>

- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In barter_integration::model

?

[Change settings](#)

Struct [barter_integration::model::Exchange](#)

Copy item path

[source](#) ·

[−]

```
pub struct Exchange(/* private fields */);
```

Expand description

Barter representation of an [Exchange](#)'s name.

eg/ `Exchange("binance_spot")`, `Exchange("bitfinex")`,
`Exchange("gateio_spot")`, etc.

Trait Implementations§

[source§](#)

impl [Clone](#) for [Exchange](#)

[source§](#)

fn [clone](#)(&self) -> [Exchange](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [Exchange](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [Exchange](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [Exchange](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<E> From<E> for Exchange
where E: Into<Cow<'static, str>>,

[source§](#)

fn from(exchange: E) -> Self

Converts to this type from the input type.

[source§](#)

impl Hash for Exchange

[source§](#)

fn hash<__H: Hasher>(&self, state: &mut __H)

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

fn hash_slice<H>(data: &[Self], state: &mut H)
where H: Hasher, Self: Sized,

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl Ord for Exchange

[source§](#)

fn cmp(&self, other: &Exchange) -> Ordering

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

fn max(self, other: Self) -> Self
where Self: Sized,

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

fn min(self, other: Self) -> Self
where Self: Sized,

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source§](#)

```
impl PartialEq for Exchange
```

[source§](#)

```
fn eq(&self, other: &Exchange) -> bool
```

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

```
impl PartialOrd for Exchange
```

[source§](#)

```
fn partial_cmp(&self, other: &Exchange) -> Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

```
fn le(&self, other: &Rhs) -> bool
```

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

```
fn gt(&self, other: &Rhs) -> bool
```

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the `>=` operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Exchange](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Eq](#) for [Exchange](#)

[source§](#)

impl [StructuralPartialEq](#) for [Exchange](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Exchange](#)

§

impl [Send](#) for [Exchange](#)

§

impl [Sync](#) for [Exchange](#)

§

impl [Unpin](#) for [Exchange](#)

§

impl [UnwindSafe](#) for [Exchange](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)

[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)

[source§](#)

**impl<T> [ToString](#) for T
where T: [Display](#) + [?Sized](#),**

[source§](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)

[source§](#)

**impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,**

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

```
fn try_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<T, U> TryInto<U> for T  
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source§](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

impl<T> [DeserializeOwned](#) for T
where T: for<'de> [Deserialize](#)<'de> ,

model - instrument

[barter_integration](#)0.5.3

[Module instrument](#)

- [Modules](#)
- [Structs](#)

[In barter_integration::model](#)

[?](#)
[Change settings](#)

Module [barter_integration::model::instrument](#)

Copy item path

[source](#) · [\[-\]](#)

Expand description

[Instrument](#) related data structures.

eg/ Instrument, InstrumentKind, OptionContract, Symbol, etc.

Modules§

- [kind](#)
- [symbol](#)

Structs§

- [Instrument](#)
Barter representation of an Instrument. Used to uniquely identify a base_quote pair, and it's associated instrument type.

barter_integration0.5.3

Instrument

Fields

- [base](#)
- [kind](#)
- [quote](#)

Methods

- [new](#)

Trait Implementations

- [Clone](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [From<\(S, S, InstrumentKind\)>](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)

- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In [barter_integration::model::instrument](#)

?

[Change settings](#)

Struct [barter_integration::model::instrument::Instrument](#)

Copy item path

[source](#) ·

[−]

```
pub struct Instrument {
    pub base: Symbol,
    pub quote: Symbol,
    pub kind: InstrumentKind,
}
```

Expand description

Barter representation of an Instrument. Used to uniquely identify a base_quote pair, and it's associated instrument type.

eg/ Instrument { base: "btc", quote: "usdt", kind: Spot }

Fields§

§base: [Symbol](#)§quote: [Symbol](#)§kind: [InstrumentKind](#)

Implementations§

[source](#)§

impl [Instrument](#)

[source](#)

```
pub fn new<S>(base: S, quote: S, kind: InstrumentKind) -> Self
where S: Into<Symbol>,
```

Constructs a new [Instrument](#) using the provided configuration.

Trait Implementations§

[source§](#)

impl [Clone](#) for [Instrument](#)

[source§](#)

fn [clone](#)(&self) -> [Instrument](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [Instrument](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [Instrument](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [Instrument](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<S> [From](#)<(S, S, [InstrumentKind](#))> for [Instrument](#)
where S: [Into](#)<[Symbol](#)>,

[source§](#)

fn [from](#)((base, quote, kind): (S, S, [InstrumentKind](#))) -> Self

Converts to this type from the input type.

[source§](#)

impl [Hash](#) for [Instrument](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)

1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)

[source§](#)

impl [Ord](#) for [Instrument](#)

[source§](#)

fn [cmp](#)(&self, other: [&Instrument](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)

1.21.0 · [source§](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)

1.21.0 · [source§](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)

1.50.0 · [source§](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source§](#)

```
impl PartialEq for Instrument
```

[source§](#)

```
fn eq(&self, other: &Instrument) -> bool
```

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

```
impl PartialOrd for Instrument
```

[source§](#)

```
fn partial_cmp(&self, other: &Instrument) -> Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

```
fn le(&self, other: &Rhs) -> bool
```

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

```
fn gt(&self, other: &Rhs) -> bool
```

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Instrument](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Eq](#) for [Instrument](#)

[source§](#)

impl [StructuralPartialEq](#) for [Instrument](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Instrument](#)

§

impl [Send](#) for [Instrument](#)

§

impl [Sync](#) for [Instrument](#)

§

impl [Unpin](#) for [Instrument](#)

§

impl [UnwindSafe](#) for [Instrument](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)

[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)

[source§](#)

**impl<T> [ToString](#) for T
where T: [Display](#) + [?Sized](#),**

[source§](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)

[source§](#)

**impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,**

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

```
fn try_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<T, U> TryInto<U> for T  
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source§](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

impl<T> [DeserializeOwned](#) for T
where T: for<'de> [Deserialize](#)<'de>,

model - instrument - kind



[barter_integration](#)0.5.3

[FutureContract](#)

[Fields](#)

- [expiry](#)

[Trait Implementations](#)

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)

- [Intro<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In [barter_integration::model::instrument::kind](#)

[?](#)
[Change settings](#)

Struct [barter_integration::model::instrument::kind::FutureContract](#)

Copy item path

[source](#) ·

```
pub struct FutureContract {
    pub expiry: DateTime<Utc>,
}
```

Expand description

Configuration of an [InstrumentKind::Future](#) contract.

Fields§

§expiry: [DateTime<Utc>](#)

Trait Implementations§

[source§](#)

impl [Clone](#) for [FutureContract](#)

[source§](#)

fn [clone](#)(&self) -> [FutureContract](#)

Returns a copy of the value. [Read more](#)
 1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [FutureContract](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [FutureContract](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Hash](#) for [FutureContract](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [FutureContract](#)

[source§](#)

fn [cmp](#)(&self, other: [&FutureContract](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

```
fn max(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source](#)

```
fn min(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source](#)

```
impl PartialEq for FutureContract
```

[source](#)

```
fn eq(&self, other: &FutureContract) -> bool
```

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source](#)

```
impl PartialOrd for FutureContract
```

[source](#)

```
fn partial\_cmp(&self, other: &FutureContract) -> Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [FutureContract](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Copy](#) for [FutureContract](#)

[source§](#)

impl [Eq](#) for [FutureContract](#)

[source§](#)

impl [StructuralPartialEq](#) for [FutureContract](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [FutureContract](#)

§

impl [Send](#) for [FutureContract](#)

[§](#)

impl [Sync](#) for [FutureContract](#)

[§](#)

impl [Unpin](#) for [FutureContract](#)

[§](#)

impl [UnwindSafe](#) for [FutureContract](#)

Blanket Implementations[§](#)

[source](#)[§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)[§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source](#)[§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

```
fn in\_current\_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

```
impl<T> Same for T
```

[§](#)

```
type Output = T
```

Should always be Self

[source§](#)

```
impl<T> ToOwned for T  
where T: Clone,
```

[§](#)

```
type Owned = T
```

The resulting type after obtaining ownership.

[source§](#)

```
fn to\_owned(&self) -> T
```

Creates owned data from borrowed data, usually by cloning. [Read more source§](#)

```
fn clone\_into(&self, target: &mut T)
```

Uses borrowed data to replace owned data, usually by cloning. [Read more source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more source](#)

impl<T> [DeserializeOwned](#) for T
where T: for<'de> [Deserialize](#)<'de>,



[barter_integration](#)0.5.3

[OptionKind](#)

[Variants](#)

- [Call](#)
- [Put](#)

[Trait Implementations](#)

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)

- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In

barter_integration::model::instrument::kind

[?](#)

[Change settings](#)

Enum barter_integration::model::instrument::kind::OptionKind

Copy item path

[source](#) ·

[\[-\]](#)

```
pub enum OptionKind {
    Call,
    Put,
}
```

Expand description

[OptionContract](#) kind - Put or Call.

Variants[§](#)

[§](#)

Call

[§](#)

Put

Trait Implementations§

[source§](#)

impl [Clone](#) for [OptionKind](#)

[source§](#)

fn [clone](#)(&self) -> [OptionKind](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [OptionKind](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [OptionKind](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [OptionKind](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [Hash](#) for [OptionKind](#)

[source§](#)

```
fn hash<__H: Hasher>(&self, state: &mut \_\_H)
```

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

```
fn hash\_slice<H>(data: &\[Self\], state: &mut H)  
where H: Hasher, Self: Sized,
```

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [OptionKind](#)

[source§](#)

```
fn cmp(&self, other: &OptionKind) -> Ordering
```

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

```
fn max(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

```
fn min(self, other: Self) -> Self  
where Self: Sized,
```

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [OptionKind](#)

[source§](#)

```
fn eq(&self, other: &OptionKind) -> bool
```

This method tests for self and other values to be equal, and is used by ==.

1.0.0 · [source§](#)

fn [ne](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.

[source§](#)

impl [PartialOrd](#) for [OptionKind](#)

[source§](#)

fn [partial_cmp](#)(&self, other: [&OptionKind](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)

1.0.0 · [source§](#)

fn [lt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)

1.0.0 · [source§](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)

1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)

1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)

[source§](#)

impl [Serialize](#) for [OptionKind](#)

[source§](#)

```
fn serialize<__S>(&self, __serializer: __S) -> Result<__S::Ok,  
__S::Error>  
where __S: Serializer,
```

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)

```
impl Copy for OptionKind
```

[source](#)

```
impl Eq for OptionKind
```

[source](#)

```
impl StructuralPartialEq for OptionKind
```

Auto Trait Implementations

[§](#)

```
impl RefUnwindSafe for OptionKind
```

[§](#)

```
impl Send for OptionKind
```

[§](#)

```
impl Sync for OptionKind
```

[§](#)

```
impl Unpin for OptionKind
```

[§](#)

```
impl UnwindSafe for OptionKind
```

Blanket Implementations

[source](#)

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self
[source](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

impl<T> [DeserializeOwned](#) for T
where T: for<'de> [Deserialize](#)<'de>,



barter_integration0.5.3

Module kind

- [Structs](#)
- [Enums](#)

In barter_integration::model::instrument

[?](#)
[Change settings](#)

Module [barter_integration::model::instrument::kind](#)

Copy item path

[source](#) ·

[-]

Structs§

- [FutureContract](#)
Configuration of an [InstrumentKind::Future](#) contract.
- [OptionContract](#)
Configuration of an [InstrumentKind::Option](#) contract.

Enums§

- [InstrumentKind](#)
Defines the type of [Instrument](#) which is being traded on a given base_quote market.
- [OptionExercise](#)
[OptionContract](#) exercise style.
- [OptionKind](#)
[OptionContract](#) kind - Put or Call.

barter_integration0.5.3

OptionExercise

Variants

- [American](#)
- [Bermudan](#)
- [European](#)

Trait Implementations

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)

- [VZip<V>](#)
- [WithSubscriber](#)

In [barter_integration::model::instrument::kind](#)

[?](#)

[Change settings](#)

Enum [barter_integration::model::instrument::kind::OptionExercise](#)

Copy item path

[source](#) ·

[−]

```
pub enum OptionExercise {  
    American,  
    Bermudan,  
    European,  
}
```

Expand description

[OptionContract](#) exercise style.

Variants§

§

American

§

Bermudan

§

European

Trait Implementations§

[source](#)§

impl [Clone](#) for [OptionExercise](#)

[source§](#)

fn [clone](#)(&self) -> [OptionExercise](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [OptionExercise](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [OptionExercise](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [OptionExercise](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [Hash](#) for [OptionExercise](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)

1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: &[\[Self\]](#), state: &mut [H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [OptionExercise](#)

[source§](#)

fn [cmp](#)(&self, other: &[OptionExercise](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source§](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source§](#)

fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [OptionExercise](#)

[source§](#)

fn [eq](#)(&self, other: &[OptionExercise](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [OptionExercise](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[OptionExercise](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)

1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)

1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)

1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)

1.0.0 · [source§](#)

fn [ge](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)

[source§](#)

impl [Serialize](#) for [OptionExercise](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::[Ok](#), __S::[Error](#)>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)

[source§](#)

impl [Copy](#) for [OptionExercise](#)

[source§](#)

impl [Eq](#) for [OptionExercise](#)

[source](#)§

impl [StructuralPartialEq](#) for [OptionExercise](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [OptionExercise](#)

§

impl [Send](#) for [OptionExercise](#)

§

impl [Sync](#) for [OptionExercise](#)

§

impl [Unpin](#) for [OptionExercise](#)

§

impl [UnwindSafe](#) for [OptionExercise](#)

Blanket Implementations§

[source](#)§

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)§

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)

[source](#)§

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source](#)§

```
fn borrow(&self) -> &T
```

Immutably borrows from an owned value. [Read more source§](#)

```
impl<T> BorrowMut<T> for T  
where T: ?Sized,
```

[source§](#)

```
fn borrow_mut(&mut self) -> &mut T
```

Mutably borrows from an owned value. [Read more source§](#)

```
impl<Q, K> Comparable<K> for Q  
where Q: Ord + ?Sized, K: Borrow<Q> + ?Sized,
```

[source§](#)

```
fn compare(&self, key: &K) -> Ordering
```

Compare self to key and return their ordering. [source§](#)

```
impl<Q, K> Equivalent<K> for Q  
where Q: Eq + ?Sized, K: Borrow<Q> + ?Sized,
```

[source§](#)

```
fn equivalent(&self, key: &K) -> bool
```

Checks if this value is equivalent to the given key. [Read more source§](#)

```
impl<Q, K> Equivalent<K> for Q  
where Q: Eq + ?Sized, K: Borrow<Q> + ?Sized,
```

[source§](#)

```
fn equivalent(&self, key: &K) -> bool
```

Compare self to key and return true if they are equal. [source§](#)

```
impl<T> From<T> for T
```

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

**impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,**

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
impl<T> DeserializeOwned for T  
where T: for<'de> Deserialize<'de>,
```



[barter_integration](#)0.5.3

[InstrumentKind](#)

[Variants](#)

- [Future](#)
- [Option](#)
- [Perpetual](#)
- [Spot](#)

Trait Implementations

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Default](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In **barter_integration::model::instrument::kind**

[?](#)

[Change settings](#)

Enum [barter_integration::model::instrument::kind::InstrumentKind](#)

Copy item path

[source](#) ·

[–]

```
pub enum InstrumentKind {  
    Spot,  
    Future(FutureContract),  
    Perpetual,  
    Option(OptionContract),  
}
```

Expand description

Defines the type of [Instrument](#) which is being traded on a given base_quote market.

Variants§

§

Spot

§

Future([FutureContract](#))

§

Perpetual

§

Option([OptionContract](#))

Trait Implementations§

[source](#)§

impl [Clone](#) for [InstrumentKind](#)

[source](#)§

fn [clone](#)(&self) -> [InstrumentKind](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [InstrumentKind](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [Default](#) for [InstrumentKind](#)

[source§](#)

fn [default](#)() -> Self

Returns the “default value” for a type. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [InstrumentKind](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::[Error](#)>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Display](#) for [InstrumentKind](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [Hash](#) for [InstrumentKind](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source](#)

impl [Ord](#) for [InstrumentKind](#)

[source](#)

fn [cmp](#)(&self, other: [&InstrumentKind](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source](#)

fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),

Restrict a value to a certain interval. [Read more](#)
[source](#)

impl [PartialEq](#) for [InstrumentKind](#)

[source](#)

fn [eq](#)(&self, other: [&InstrumentKind](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source](#)

fn [ne](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.

[source§](#)

impl [PartialOrd](#) for [InstrumentKind](#)

[source§](#)

fn [partial_cmp](#)(&self, other: [&InstrumentKind](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)

1.0.0 · [source§](#)

fn [lt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)

1.0.0 · [source§](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)

1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)

1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)

[source§](#)

impl [Serialize](#) for [InstrumentKind](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)

[source§](#)

impl [Copy](#) for [InstrumentKind](#)

[source§](#)

impl [Eq](#) for [InstrumentKind](#)

[source§](#)

impl [StructuralPartialEq](#) for [InstrumentKind](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [InstrumentKind](#)

§

impl [Send](#) for [InstrumentKind](#)

§

impl [Sync](#) for [InstrumentKind](#)

§

impl [Unpin](#) for [InstrumentKind](#)

§

impl [UnwindSafe](#) for [InstrumentKind](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

**impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,**

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

impl<T> [ToOwned](#) for T
where T: [Clone](#),

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.
[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source§](#)

impl<T> [ToString](#) for T
where T: [Display](#) + ?[Sized](#),

[source§](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)
[source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source§](#)

```
impl<T, U> TryInto<U> for T
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.
[source§](#)

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.
[source§](#)

```
impl<V, T> VZip<V> for T
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

```
impl<T> DeserializeOwned for T
where T: for<'de> Deserialize<'de>,
```



barter_integration0.5.3

OptionContract

Fields

- [exercise](#)
- [expiry](#)
- [kind](#)
- [strike](#)

Trait Implementations

- [Clone](#)
- [Copy](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Eq](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)

- [WithSubscriber](#)

In [barter_integration::model::instrument::kind](#)

?

[Change settings](#)

Struct [barter_integration::model::instrument::kind::OptionContract](#)

Copy item path

[source](#) ·

[−]

```
pub struct OptionContract {  
    pub kind: OptionKind,  
    pub exercise: OptionExercise,  
    pub expiry: DateTime<Utc>,  
    pub strike: Decimal,  
}
```

Expand description

Configuration of an [InstrumentKind::Option](#) contract.

Fields§

[\\$kind](#): [OptionKind](#)[\\$exercise](#): [OptionExercise](#)[\\$expiry](#): [DateTime<Utc>](#)[\\$strike](#): [Decimal](#)

Trait Implementations§

[source](#)§

impl [Clone](#) for [OptionContract](#)

[source](#)§

fn [clone](#)(&self) -> [OptionContract](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source](#)§

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [OptionContract](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl<'de> [Deserialize](#)<'de> for [OptionContract](#)

[source§](#)

fn [deserialize](#)<__D>(__deserializer: __D) -> [Result](#)<Self, __D::Error>
where __D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)
[source§](#)

impl [Hash](#) for [OptionContract](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)
1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)
[source§](#)

impl [Ord](#) for [OptionContract](#)

[source§](#)

fn [cmp](#)(&self, other: [&OptionContract](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source§](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source](#)

fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),

Restrict a value to a certain interval. [Read more](#)
[source](#)

impl [PartialEq](#) for [OptionContract](#)

[source](#)

fn [eq](#)(&self, other: &[OptionContract](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source](#)

impl [PartialOrd](#) for [OptionContract](#)

[source](#)

fn [partial_cmp](#)(&self, other: &[OptionContract](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [OptionContract](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Copy](#) for [OptionContract](#)

[source§](#)

impl [Eq](#) for [OptionContract](#)

[source§](#)

impl [StructuralPartialEq](#) for [OptionContract](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [OptionContract](#)

§

impl [Send](#) for [OptionContract](#)

[§](#)

impl [Sync](#) for [OptionContract](#)

[§](#)

impl [Unpin](#) for [OptionContract](#)

[§](#)

impl [UnwindSafe](#) for [OptionContract](#)

Blanket Implementations[§](#)

[source](#)[§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)[§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source](#)[§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

```
fn in\_current\_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

```
impl<T> Same for T
```

[§](#)

```
type Output = T
```

Should always be Self

[source§](#)

```
impl<T> ToOwned for T  
where T: Clone,
```

[§](#)

```
type Owned = T
```

The resulting type after obtaining ownership.

[source§](#)

```
fn to\_owned(&self) -> T
```

Creates owned data from borrowed data, usually by cloning. [Read more source§](#)

```
fn clone\_into(&self, target: &mut T)
```

Uses borrowed data to replace owned data, usually by cloning. [Read more source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more source§](#)

```
impl<T> DeserializeOwned for T
where T: for<'de> Deserialize<'de>,
```

model - instrument - symbol

[barter_integration](#)0.5.3

[Module symbol](#)

- [Structs](#)

[In barter_integration::model::instrument](#)

[?](#)
[Change settings](#)

Module [barter_integration::model::instrument::symbol](#)

Copy item path

[source](#) ·

[-]

Structs§

- [Symbol](#)
Barter new type representing a currency symbol String identifier.

barter_integration0.5.3

Symbol

Methods

- [new](#)

Trait Implementations

- [AsRef<str>](#)
- [Clone](#)
- [Debug](#)
- [Deserialize<'de>](#)
- [Display](#)
- [Eq](#)
- [From<S>](#)
- [Hash](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [DeserializeOwned](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)

- [WithSubscriber](#)

In

barter_integration::model::instrument::symbol

[?](#)

[Change settings](#)

Struct barter_integration::model::instrument::symbol::Symbol

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct Symbol(/* private fields */);
```

Expand description

Barter new type representing a currency symbol String identifier.

eg/ "btc", "eth", "usdt", etc

Implementations§

[source§](#)

impl Symbol

[source](#)

```
pub fn new<S>(input: S) -> Self  
where S: Into<String>,
```

Construct a new [Symbol](#) new type using the provided Into<Symbol> value.

Trait Implementations§

[source§](#)

impl AsRef<str**> for Symbol**

[source§](#)

fn [as_ref](#)(&self) -> [&str](#)

Converts this type into a shared reference of the (usually inferred) input type.

[source§](#)

impl [Clone](#) for [Symbol](#)

[source§](#)

fn [clone](#)(&self) -> [Symbol](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)

[source§](#)

impl [Debug](#) for [Symbol](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<'de> [Deserialize](#)<'de> for [Symbol](#)

[source§](#)

fn [deserialize](#)<D>(deserializer: D) -> [Result](#)<Self, D::Error>
where D: [Deserializer](#)<'de>,

Deserialize this value from the given Serde deserializer. [Read more](#)

[source§](#)

impl [Display](#) for [Symbol](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl<S> [From](#)<S> for [Symbol](#)
where S: [Into](#)<[String](#)>,

[source§](#)

fn [from](#)(input: S) -> Self

Converts to this type from the input type.

[source§](#)

impl [Hash](#) for [Symbol](#)

[source§](#)

fn [hash](#)<__H: [Hasher](#)>(&self, state: [&mut __H](#))

Feeds this value into the given [Hasher](#). [Read more](#)

1.3.0 · [source§](#)

fn [hash_slice](#)<H>(data: [&\[Self\]](#), state: [&mut H](#))
where H: [Hasher](#), Self: [Sized](#),

Feeds a slice of this type into the given [Hasher](#). [Read more](#)

[source§](#)

impl [Ord](#) for [Symbol](#)

[source§](#)

fn [cmp](#)(&self, other: [&Symbol](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)

1.21.0 · [source§](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)

1.21.0 · [source§](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)

1.50.0 · [source§](#)

fn [clamp](#)(self, min: Self, max: Self) -> Self
where Self: [Sized](#) + [PartialOrd](#),

Restrict a value to a certain interval. [Read more](#)
[source§](#)

impl [PartialEq](#) for [Symbol](#)

[source§](#)

fn [eq](#)(&self, other: &[Symbol](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [Symbol](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[Symbol](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the `>=` operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Symbol](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::[Ok](#), __S::[Error](#)>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Eq](#) for [Symbol](#)

[source§](#)

impl [StructuralPartialEq](#) for [Symbol](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Symbol](#)

§

impl [Send](#) for [Symbol](#)

§

impl [Sync](#) for [Symbol](#)

§

impl [Unpin](#) for [Symbol](#)

§

impl [UnwindSafe](#) for [Symbol](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)

[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)

[source§](#)

**impl<T> [ToString](#) for T
where T: [Display](#) + [?Sized](#),**

[source§](#)

default fn [to_string](#)(&self) -> [String](#)

Converts the given value to a String. [Read more](#)

[source§](#)

**impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,**

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

```
fn try_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<T, U> TryInto<U> for T  
where U: TryFrom<T>,
```

[§](#)

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source§](#)

```
fn try_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source§](#)

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source§](#)

```
fn vzip(self) -> V
```

[source§](#)

```
impl<T> WithSubscriber for T
```

[source§](#)

```
fn with_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

```
fn with_current_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source§](#)

impl<T> [DeserializeOwned](#) for T
where T: for<'de> [Deserialize](#)<'de>,

de

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)
[Change settings](#)

Function [barter_integration::de::de_str_f64_epoch_ms_as_datetime_utc](#)

Copy item path

[source](#) ·

[-]

```
pub fn de_str_f64_epoch_ms_as_datetime_utc<'de, D>(
    deserializer: D
) -> Result<DateTime<Utc>, D::Error>
where
    D: Deserializer<'de>,
```

Expand description

Deserialize a &str “f64” milliseconds value as [DateTime](#)<[Utc](#)>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)
[Change settings](#)

Function [barter_integration::de::de_str](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub fn de_str<'de, D, T>(deserializer: D) -> Result<T, D::Error>
where
    D: Deserializer<'de>,
    T: FromStr,
    T::Err: Display,
```

Expand description

Deserialize a String as the desired type.

[barter_integration](#)0.5.3

[Module de](#)

- [Functions](#)

[In crate barter_integration](#)

[?](#)

[Change settings](#)

Module [barter_integration::de](#)

Copy item path

[source](#) ·

[\[-\]](#)

Expand description

Utilities to assist deserialisation.

Functions§

- [datetime_utc_from_epoch_duration](#)
Determine the DateTime<Utc> from the provided Duration since the epoch.
- [de_str](#)
Deserialize a String as the desired type.
- [de_str_f64_epoch_ms_as_datetime_utc](#)
Deserialize a &str “f64” milliseconds value as DateTime<Utc>.

- [de_str_f64_epoch_s_as_datetime_utc](#)
Deserialize a &str “f64” seconds value as DateTime<Utc>.
- [de_str_u64_epoch_ms_as_datetime_utc](#)
Deserialize a &str “u64” milliseconds value as DateTime<Utc>.
- [de_u64_epoch_ms_as_datetime_utc](#)
Deserialize a u64 milliseconds value as DateTime<Utc>.
- [extract_next](#)
Assists deserialisation of sequences by attempting to extract & parse the next element in the provided sequence.
- [se_element_to_vector](#)
Serialize a generic element T as a Vec<T>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)
[Change settings](#)

Function [barter_integration::de::de_u64_epoch_ms_as_datetime_utc](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub fn de_u64_epoch_ms_as_datetime_utc<'de, D>(
    deserializer: D
) -> Result<DateTime<Utc>, D::Error>
where
    D: Deserializer<'de>,
```

Expand description

Deserialize a u64 milliseconds value as DateTime<Utc>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)

[Change settings](#)

Function [barter_integration::de::extract_next](#)

Copy item path

[source](#) ·

```
pub fn extract_next<'de, SeqAccessor, Target>(
    sequence: &mut SeqAccessor,
    name: &'static str
) -> Result<Target, SeqAccessor::Error>
where
    SeqAccessor: SeqAccess<'de>,
    Target: DeserializeOwned,
```

Expand description

Assists deserialisation of sequences by attempting to extract & parse the next element in the provided sequence.

A [serde::de::Error](#) is returned if the element does not exist, or it cannot be deserialized into the Target type inferred.

Example sequence:

```
[“20180.30000”,“0.00010000”,“1661978265.280067”,“s”,“l”,“”]
```

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)

[Change settings](#)

Function [barter_integration::de::se_element_to_vector](#)

Copy item path

[source](#) ·

```
pub fn se_element_to_vector<T, S>(
    element: T,
    serializer: S
) -> Result<S::Ok, S::Error>
```

where
S: [Serializer](#),
T: [Serialize](#),

Expand description

Serialize a generic element T as a Vec<T>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)
[Change settings](#)

Function [barter_integration::de::de_str_f64_epoch_s_as_datetime_utc](#)

Copy item path

[source](#) ·

[-]

```
pub fn de_str_f64_epoch_s_as_datetime_utc<'de, D>(
    deserializer: D
) -> Result<DateTime<Utc>, D::Error>
where
    D: Deserializer<'de>,
```

Expand description

Deserialize a &str “f64” seconds value as [DateTime](#)<[Utc](#)>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)
[Change settings](#)

Function [barter_integration::de::de_str_u64_epoch_ms_as_datetime_utc](#)

Copy item path

[source](#) ·

```
pub fn de_str_u64_epoch_ms_as_datetime_utc<'de, D>(
    deserializer: D
) -> Result<DateTime<Utc>, D::Error>
where
    D: Deserializer<'de>,
```

Expand description

Deserialize a &str “u64” milliseconds value as [DateTime](#)<[Utc](#)>.

[barter_integration](#)0.5.3

[In barter_integration::de](#)

[?](#)

[Change settings](#)

Function [barter_integration::de::datetime_utc_from_epoch_duration](#)

Copy item path

[source](#) ·

```
pub fn datetime_utc_from_epoch_duration(duration: Duration) -> DateTime<Utc>
```

Expand description

Determine the [DateTime](#)<[Utc](#)> from the provided [Duration](#) since the epoch.

metric

barter_integration0.5.3

Metric

Fields

- [fields](#)
- [name](#)
- [tags](#)
- [time](#)

Trait Implementations

- [Clone](#)
- [Debug](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In barter_integration::metric

[?](#)

[Change settings](#)

Struct [barter_integration::metric::Metric](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct Metric {  
    pub name: &'static str,  
    pub time: u64,  
    pub tags: Vec<Tag>,  
    pub fields: Vec<Field>,  
}
```

Fields§

[\\$name](#): &'static [str](#)

Metric name.

[\\$time](#): [u64](#)

Milliseconds since the Unix epoch.

[\\$tags](#): [Vec](#)<[Tag](#)>

Key-Value pairs to categorise the Metric.

[\\$fields](#): [Vec](#)<[Field](#)>

Observed measurements.

Trait Implementations§

[source](#)§

impl [Clone](#) for [Metric](#)

[source](#)§

fn [clone](#)(&self) -> [Metric](#)

Returns a copy of the value. [Read more](#)

1.0.0 · [source](#)§

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)

[source](#)§

impl [Debug](#) for [Metric](#)

[source§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source§](#)

impl [PartialEq](#) for [Metric](#)

[source§](#)

fn [eq](#)(&self, other: &[Metric](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [Metric](#)

[source§](#)

fn [partial_cmp](#)(&self, other: &[Metric](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: &[Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source](#)§

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source](#)§

impl [Serialize](#) for [Metric](#)

[source](#)§

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source](#)§

impl [StructuralPartialEq](#) for [Metric](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Metric](#)

§

impl [Send](#) for [Metric](#)

§

impl [Sync](#) for [Metric](#)

§

impl [Unpin](#) for [Metric](#)

§

impl [UnwindSafe](#) for [Metric](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

**impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,**

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[barter_integration](#)0.5.3

[Module metric](#)

- [Structs](#)
- [Enums](#)

[In crate barter_integration](#)

[?](#)

[Change settings](#)

Module [barter_integration::metric](#)

Copy item path

[source](#) · [\[-\]](#)

Expand description

Contains the flexible Metric type used for representing real-time metrics generically.

Structs[§](#)

- [Field](#)
- [Metric](#)
- [Tag](#)

Enums

- [Value](#)



[barter_integration](#)0.5.3

[Tag](#)

[Fields](#)

- [key](#)
- [value](#)

[Methods](#)

- [new](#)

[Trait Implementations](#)

- [Clone](#)
- [Debug](#)
- [Eq](#)
- [From<\(&'static str, S\)>](#)
- [Ord](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

[Auto Trait Implementations](#)

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

[Blanket Implementations](#)

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [Comparable<K>](#)
- [Equivalent<K>](#)
- [Equivalent<K>](#)
- [From<T>](#)
- [Instrument](#)

- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

[In barter_integration::metric](#)

[?](#)

[Change settings](#)

Struct [barter_integration::metric::Tag](#)

Copy item path

[source](#) ·

[\[-\]](#)

```
pub struct Tag {
    pub key: &'static str,
    pub value: String,
}
```

Fields§

[§](#)key: &'static [str](#)[§](#)value: [String](#)

Implementations§

[source](#)[§](#)

impl [Tag](#)

[source](#)

```
pub fn new<S>(key: &'static str, value: S) -> Self
where S: Into<String>,
```

Trait Implementations§

[source](#)[§](#)

impl [Clone](#) for [Tag](#)

[source](#)[§](#)

fn [clone](#)(&self) -> [Tag](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source](#)

impl [Debug](#) for [Tag](#)

[source](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source](#)

impl<S> [From](#)<(&'static [str](#), S)> for [Tag](#)
where S: [Into](#)<[String](#)>,

[source](#)

fn [from](#)((key, value): (&'static [str](#), S)) -> Self

Converts to this type from the input type.
[source](#)

impl [Ord](#) for [Tag](#)

[source](#)

fn [cmp](#)(&self, other: &[Tag](#)) -> [Ordering](#)

This method returns an [Ordering](#) between self and other. [Read more](#)
1.21.0 · [source](#)

fn [max](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the maximum of two values. [Read more](#)
1.21.0 · [source](#)

fn [min](#)(self, other: Self) -> Self
where Self: [Sized](#),

Compares and returns the minimum of two values. [Read more](#)
1.50.0 · [source](#)

```
fn clamp(self, min: Self, max: Self) -> Self  
where Self: Sized + PartialOrd,
```

Restrict a value to a certain interval. [Read more](#)
[source§](#)

```
impl PartialEq for Tag
```

[source§](#)

```
fn eq(&self, other: &Tag) -> bool
```

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

```
impl PartialOrd for Tag
```

[source§](#)

```
fn partial_cmp(&self, other: &Tag) -> Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

```
fn le(&self, other: &Rhs) -> bool
```

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

```
fn gt(&self, other: &Rhs) -> bool
```

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Tag](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::[Ok](#), __S::[Error](#)>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [Eq](#) for [Tag](#)

[source§](#)

impl [StructuralPartialEq](#) for [Tag](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Tag](#)

§

impl [Send](#) for [Tag](#)

§

impl [Sync](#) for [Tag](#)

§

impl [Unpin](#) for [Tag](#)

§

impl [UnwindSafe](#) for [Tag](#)

Blanket Implementations§

[source§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<Q, K> [Comparable](#)<K> for Q
where Q: [Ord](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [compare](#)(&self, key: [&K](#)) -> [Ordering](#)

Compare self to key and return their ordering.
[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Checks if this value is equivalent to the given key. [Read more](#)

[source§](#)

impl<Q, K> [Equivalent](#)<K> for Q
where Q: [Eq](#) + ?[Sized](#), K: [Borrow](#)<Q> + ?[Sized](#),

[source§](#)

fn [equivalent](#)(&self, key: [&K](#)) -> [bool](#)

Compare self to key and return true if they are equal.

[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self

[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)

[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)

[source§](#)

**impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,**

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.

[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.

[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source§](#)

impl<T> [WithSubscriber](#) for T

[source§](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source§](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)



barter_integration0.5.3

Field

Fields

- [key](#)
- [value](#)

Methods

- [new](#)

Trait Implementations

- [Clone](#)
- [Debug](#)
- [From<\(&'static str, S\)>](#)
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In barter_integration::metric

--

[?](#)
[Change settings](#)

Struct [barter_integration::metric::Field](#)

Copy item path

[source](#) · [\[-\]](#)

```
pub struct Field {  
    pub key: &'static str,  
    pub value: Value,  
}
```

Fields§

[§key](#): &'static [str](#)[§value](#): [Value](#)

Implementations§

[source§](#)

impl [Field](#)

[source](#)

```
pub fn new<S>(key: &'static str, value: S) -> Self  
where S: Into<Value>,
```

Trait Implementations§

[source§](#)

impl [Clone](#) for [Field](#)

[source§](#)

```
fn clone(&self) -> Field
```

Returns a copy of the value. [Read more](#)
1.0.0 · [source§](#)

```
fn clone\_from(&mut self, source: Self)
```

Performs copy-assignment from source. [Read more](#)
[source§](#)

impl [Debug](#) for [Field](#)

[source§](#)

```
fn fmt(&self, f: &mut Formatter<'_>) -> Result
```

Formats the value using the given formatter. [Read more](#)

[source§](#)

```
impl<S> From<(&'static str, S)> for Field  
where S: Into<Value>,
```

[source§](#)

```
fn from((key, value): (&'static str, S)) -> Self
```

Converts to this type from the input type.

[source§](#)

impl [PartialEq](#) for [Field](#)

[source§](#)

```
fn eq(&self, other: &Field) -> bool
```

This method tests for self and other values to be equal, and is used by ==.

1.0.0 · [source§](#)

```
fn ne(&self, other: &Rhs) -> bool
```

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.

[source§](#)

impl [PartialOrd](#) for [Field](#)

[source§](#)

```
fn partial\_cmp(&self, other: &Field) -> Option<Ordering>
```

This method returns an ordering between self and other values if one exists. [Read more](#)

1.0.0 · [source§](#)

```
fn lt(&self, other: &Rhs) -> bool
```

This method tests less than (for self and other) and is used by the < operator. [Read more](#)

1.0.0 · [source§](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)
[source§](#)

impl [Serialize](#) for [Field](#)

[source§](#)

fn [serialize](#)<__S>(&self, __serializer: __S) -> [Result](#)<__S::Ok, __S::Error>
where __S: [Serializer](#),

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [StructuralPartialEq](#) for [Field](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Field](#)

§

impl [Send](#) for [Field](#)

§

impl [Sync](#) for [Field](#)

§

impl [Unpin](#) for [Field](#)

[§](#)

impl [UnwindSafe](#) for [Field](#)

Blanket Implementations[§](#)

[source](#)[§](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source](#)[§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source](#)[§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source](#)[§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source](#)[§](#)

impl<T> [From](#)<T> for T

[source](#)[§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

**impl<T, U> [Into](#)<U> for T
where U: [From](#)<T>,**

[source§](#)

fn [into](#)(self) -> U

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source§](#)

impl<T> [Same](#) for T

[§](#)

type [Output](#) = T

Should always be Self
[source§](#)

**impl<T> [ToOwned](#) for T
where T: [Clone](#),**

[§](#)

type [Owned](#) = T

The resulting type after obtaining ownership.

[source§](#)

fn [to_owned](#)(&self) -> T

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source§](#)

fn [clone_into](#)(&self, target: [&mut T](#))

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source§](#)

impl<T, U> [TryFrom](#)<U> for T
where U: [Into](#)<T>,

[§](#)

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source§](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source§](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source§](#)

fn [vzip](#)(self) -> V

[source](#)

impl<T> [WithSubscriber](#) for T

[source](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[barter_integration](#)0.5.3

[Value](#)

[Variants](#)

- [Bool](#)
- [Float](#)
- [Int](#)
- [String](#)
- [UInt](#)

[Trait Implementations](#)

- [Clone](#)
- [Debug](#)
- [Deserialize](#)<'de>
- [From](#)<String>
- [From](#)<bool>
- [From](#)<f64>
- [From](#)<i64>
- [From](#)<u64>
- [PartialEq](#)
- [PartialOrd](#)
- [Serialize](#)
- [StructuralPartialEq](#)

Auto Trait Implementations

- [RefUnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)
- [UnwindSafe](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [DeserializeOwned](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToOwned](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In `barter_integration::metric`

[?](#)

[Change settings](#)

Enum `barter_integration::metric::Value`

Copy item path

[source](#) ·

[-]

```
pub enum Value {
    Float(f64),
    Int(i64),
    UInt(u64),
    Bool(bool),
    String(String),
}
```

Variants§

[§](#)

Float([f64](#))

[§](#)

Int([i64](#))

[§](#)

UInt([u64](#))

[§](#)

Bool([bool](#))

[§](#)

String([String](#))

Trait Implementations[§](#)

[source](#)[§](#)

impl [Clone](#) for [Value](#)

[source](#)[§](#)

fn [clone](#)(&self) -> [Value](#)

Returns a copy of the value. [Read more](#)
1.0.0 · [source](#)[§](#)

fn [clone_from](#)(&mut self, source: [&Self](#))

Performs copy-assignment from source. [Read more](#)
[source](#)[§](#)

impl [Debug](#) for [Value](#)

[source](#)[§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_,>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source](#)[§](#)

impl<'de> [Deserialize](#)<'de> for [Value](#)

[source](#)[§](#)

```
fn deserialize<__D>(__deserializer: __D) -> Result<Self, __D::Error>
where __D: Deserializer<'de>,
```

Deserialize this value from the given Serde deserializer. [Read more source§](#)

```
impl From<String> for Value
```

[source§](#)

```
fn from(value: String) -> Self
```

Converts to this type from the input type.
[source§](#)

```
impl From<bool> for Value
```

[source§](#)

```
fn from(value: bool) -> Self
```

Converts to this type from the input type.
[source§](#)

```
impl From<f64> for Value
```

[source§](#)

```
fn from(value: f64) -> Self
```

Converts to this type from the input type.
[source§](#)

```
impl From<i64> for Value
```

[source§](#)

```
fn from(value: i64) -> Self
```

Converts to this type from the input type.
[source§](#)

```
impl From<u64> for Value
```

[source§](#)

```
fn from(value: u64) -> Self
```

Converts to this type from the input type.

[source§](#)

impl [PartialEq](#) for [Value](#)

[source§](#)

fn [eq](#)(&self, other: [&Value](#)) -> [bool](#)

This method tests for self and other values to be equal, and is used by ==.
1.0.0 · [source§](#)

fn [ne](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests for !=. The default implementation is almost always sufficient, and should not be overridden without very good reason.
[source§](#)

impl [PartialOrd](#) for [Value](#)

[source§](#)

fn [partial_cmp](#)(&self, other: [&Value](#)) -> [Option](#)<[Ordering](#)>

This method returns an ordering between self and other values if one exists. [Read more](#)
1.0.0 · [source§](#)

fn [lt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than (for self and other) and is used by the < operator. [Read more](#)
1.0.0 · [source§](#)

fn [le](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests less than or equal to (for self and other) and is used by the <= operator. [Read more](#)
1.0.0 · [source§](#)

fn [gt](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than (for self and other) and is used by the > operator. [Read more](#)
1.0.0 · [source§](#)

fn [ge](#)(&self, other: [&Rhs](#)) -> [bool](#)

This method tests greater than or equal to (for self and other) and is used by the >= operator. [Read more](#)

[source§](#)

impl [Serialize](#) for [Value](#)

[source§](#)

```
fn serialize<__S>(&self, __serializer: __S) -> Result<__S::Ok,  
__S::Error>  
where __S: Serializer,
```

Serialize this value into the given Serde serializer. [Read more](#)
[source§](#)

impl [StructuralPartialEq](#) for [Value](#)

Auto Trait Implementations§

§

impl [RefUnwindSafe](#) for [Value](#)

§

impl [Send](#) for [Value](#)

§

impl [Sync](#) for [Value](#)

§

impl [Unpin](#) for [Value](#)

§

impl [UnwindSafe](#) for [Value](#)

Blanket Implementations§

[source§](#)

```
impl<T> Any for T  
where T: 'static + ?Sized,
```

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

fn [in_current_span](#)(self) -> [Instrumented](#)<Self>

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)

[source§](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source§](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of `From<T>` for `U` chooses to do.

[source§](#)

```
impl<T> Same for T
```

§

```
type Output = T
```

Should always be `Self`

[source§](#)

```
impl<T> ToOwned for T  
where T: Clone,
```

§

```
type Owned = T
```

The resulting type after obtaining ownership.

[source§](#)

```
fn to_owned(&self) -> T
```

Creates owned data from borrowed data, usually by cloning. [Read more](#)
[source§](#)

```
fn clone_into(&self, target: &mut T)
```

Uses borrowed data to replace owned data, usually by cloning. [Read more](#)
[source§](#)

```
impl<T, U> TryFrom<U> for T  
where U: Into<T>,
```

§

type [Error](#) = [Infallible](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_from](#)(value: U) -> [Result](#)<T, <T as [TryFrom](#)<U>>::[Error](#)>

Performs the conversion.
[source](#)

impl<T, U> [TryInto](#)<U> for T
where U: [TryFrom](#)<T>,

[§](#)

type [Error](#) = <U as [TryFrom](#)<T>>::[Error](#)

The type returned in the event of a conversion error.
[source](#)

fn [try_into](#)(self) -> [Result](#)<U, <U as [TryFrom](#)<T>>::[Error](#)>

Performs the conversion.
[source](#)

impl<V, T> [VZip](#)<V> for T
where V: [MultiLane](#)<T>,

[source](#)

fn [vzip](#)(self) -> V

[source](#)

impl<T> [WithSubscriber](#) for T

[source](#)

fn [with_subscriber](#)<S>(self, subscriber: S) -> [WithDispatch](#)<Self>
where S: [Into](#)<[Dispatch](#)>,

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)
[source](#)

fn [with_current_subscriber](#)(self) -> [WithDispatch](#)<Self>

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source](#)

```
impl<T> DeserializeOwned for T  
where T: for<'de> Deserialize<'de>,
```

error



[barter_integration](#)0.5.3

[SocketError](#)

[Variants](#)

- [Deserialise](#)
- [DeserialiseBinary](#)
- [Exchange](#)
- [Http](#)
- [HttpResponse](#)
- [HttpTimeout](#)
- [QueryParams](#)
- [Serialise](#)
- [Sink](#)
- [Subscribe](#)
- [Terminated](#)
- [Unidentifiable](#)
- [Unsupported](#)
- [UrlParse](#)
- [WebSocket](#)

[Trait Implementations](#)

- [Debug](#)
- [Display](#)
- [Error](#)
- [From<Error>](#)
- [From<Error>](#)
- [From<Error>](#)
- [From<ParseError>](#)

[Auto Trait Implementations](#)

- [!RefUnwindSafe](#)
- [!UnwindSafe](#)
- [Send](#)
- [Sync](#)
- [Unpin](#)

Blanket Implementations

- [Any](#)
- [Borrow<T>](#)
- [BorrowMut<T>](#)
- [From<T>](#)
- [Instrument](#)
- [Into<U>](#)
- [Same](#)
- [ToString](#)
- [TryFrom<U>](#)
- [TryInto<U>](#)
- [VZip<V>](#)
- [WithSubscriber](#)

In barter_integration::error

[?](#)

[Change settings](#)

Enum barter_integration::error::SocketError

Copy item path

[source](#) ·

[\[-\]](#)

```
pub enum SocketError {  
  Show 15 variants  
  Sink,  
  Deserialise {  
    error: Error,  
    payload: String,  
  },  
  DeserialiseBinary {  
    error: Error,  
    payload: Vec<u8>,  
  },  
  Serialise(Error),  
  QueryParams(Error),  
  UrlParse(ParseError),  
  Subscribe(String),  
  Terminated(String),  
  Unsupported {  
    entity: &'static str,  
    item: String,  
  },  
  WebSocket(Error),  
  Http(Error),  
  HttpTimeout(Error),  
}
```

```
    HttpResponse(StatusCode, String),  
    Unidentifiable(SubscriptionId),  
    Exchange(String),  
}
```

Expand description

All socket IO related errors generated in barter-integration.

Variants§

§

Sink

§

Deserialise

Fields

```
§error: Error  
§payload: String  
§
```

DeserialiseBinary

Fields

```
§error: Error  
§payload: Vec<u8>  
§
```

Serialise([Error](#))

§

QueryParams([Error](#))

§

UrlParse([ParseError](#))

§

Subscribe([String](#))

§

Terminated([String](#))

[§](#)

Unsupported

Fields

[§](#)entity: &'static [str](#)

[§](#)item: [String](#)

[§](#)

WebSocket([Error](#))

[§](#)

Http([Error](#))

[§](#)

HttpTimeout([Error](#))

[§](#)

HttpResponse([StatusCode](#), [String](#))

REST http response error

[§](#)

Unidentifiable([SubscriptionId](#))

[§](#)

Exchange([String](#))

Trait Implementations[§](#)

[source](#)[§](#)

impl [Debug](#) for [SocketError](#)

[source](#)[§](#)

fn [fmt](#)(&self, f: &mut [Formatter](#)<'_>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)
[source](#)[§](#)

impl [Display](#) for [SocketError](#)

[source§](#)

fn [fmt](#)(&self, __formatter: &mut [Formatter](#)<'>) -> [Result](#)

Formats the value using the given formatter. [Read more](#)

[source§](#)

impl [Error](#) for [SocketError](#)

[source§](#)

fn [source](#)(&self) -> [Option](#)<&(dyn [Error](#) + 'static)>

The lower-level source of this error, if any. [Read more](#)

1.0.0 · [source§](#)

fn [description](#)(&self) -> &str

Deprecated since 1.42.0: use the Display impl or to_string()

[Read more](#)

1.0.0 · [source§](#)

fn [cause](#)(&self) -> [Option](#)<&dyn [Error](#)>

Deprecated since 1.33.0: replaced by Error::source, which can support downcasting

[source§](#)

fn [provide](#)<'a>(&'a self, request: &mut [Request](#)<'a>)

This is a nightly-only experimental API. (error_generic_member_access)

Provides type based access to context intended for error reports. [Read more](#)

[source§](#)

impl [From](#)<[Error](#)> for [SocketError](#)

[source§](#)

fn [from](#)(error: [Error](#)) -> Self

Converts to this type from the input type.

[source§](#)

impl [From](#)<[Error](#)> for [SocketError](#)

[source§](#)

fn [from](#)(source: [Error](#)) -> Self

Converts to this type from the input type.
[source](#)

impl [From](#)<[Error](#)> for [SocketError](#)

[source](#)

fn [from](#)(source: [Error](#)) -> Self

Converts to this type from the input type.
[source](#)

impl [From](#)<[ParseError](#)> for [SocketError](#)

[source](#)

fn [from](#)(source: [ParseError](#)) -> Self

Converts to this type from the input type.

Auto Trait Implementations

[§](#)

impl ![RefUnwindSafe](#) for [SocketError](#)

[§](#)

impl [Send](#) for [SocketError](#)

[§](#)

impl [Sync](#) for [SocketError](#)

[§](#)

impl [Unpin](#) for [SocketError](#)

[§](#)

impl ![UnwindSafe](#) for [SocketError](#)

Blanket Implementations

[source](#)

impl<T> [Any](#) for T
where T: 'static + ?[Sized](#),

[source§](#)

fn [type_id](#)(&self) -> [TypeId](#)

Gets the TypeId of self. [Read more](#)
[source§](#)

impl<T> [Borrow](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow](#)(&self) -> [&T](#)

Immutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [BorrowMut](#)<T> for T
where T: ?[Sized](#),

[source§](#)

fn [borrow_mut](#)(&mut self) -> [&mut T](#)

Mutably borrows from an owned value. [Read more](#)
[source§](#)

impl<T> [From](#)<T> for T

[source§](#)

fn [from](#)(t: T) -> T

Returns the argument unchanged.

[source§](#)

impl<T> [Instrument](#) for T

[source§](#)

fn [instrument](#)(self, span: [Span](#)) -> [Instrumented](#)<Self>

Instruments this type with the provided [Span](#), returning an Instrumented wrapper. [Read more](#)
[source§](#)

```
fn in\_current\_span(self) -> Instrumented<Self>
```

Instruments this type with the [current Span](#), returning an Instrumented wrapper. [Read more](#)
[source](#)

```
impl<T, U> Into<U> for T  
where U: From<T>,
```

[source](#)

```
fn into(self) -> U
```

Calls `U::from(self)`.

That is, this conversion is whatever the implementation of [From](#)<T> for U chooses to do.

[source](#)

```
impl<T> Same for T
```

§

```
type Output = T
```

Should always be Self
[source](#)

```
impl<T> ToString for T  
where T: Display + ?Sized,
```

[source](#)

```
default fn to\_string(&self) -> String
```

Converts the given value to a String. [Read more](#)
[source](#)

```
impl<T, U> TryFrom<U> for T  
where U: Into<T>,
```

§

```
type Error = Infallible
```

The type returned in the event of a conversion error.
[source](#)

```
fn try\_from(value: U) -> Result<T, <T as TryFrom<U>>::Error>
```

Performs the conversion.

[source](#)§

```
impl<T, U> TryInto<U> for T  
where U: TryFrom<T>,
```

§

```
type Error = <U as TryFrom<T>>::Error
```

The type returned in the event of a conversion error.

[source](#)§

```
fn try\_into(self) -> Result<U, <U as TryFrom<T>>::Error>
```

Performs the conversion.

[source](#)§

```
impl<V, T> VZip<V> for T  
where V: MultiLane<T>,
```

[source](#)§

```
fn vzip(self) -> V
```

[source](#)§

```
impl<T> WithSubscriber for T
```

[source](#)§

```
fn with\_subscriber<S>(self, subscriber: S) -> WithDispatch<Self>  
where S: Into<Dispatch>,
```

Attaches the provided [Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)

[source](#)§

```
fn with\_current\_subscriber(self) -> WithDispatch<Self>
```

Attaches the current [default Subscriber](#) to this type, returning a [WithDispatch](#) wrapper. [Read more](#)



[barter_integration](#)0.5.3

[Module error](#)

- [Enums](#)

[In crate barter_integration](#)

?

[Change settings](#)

Module [barter_integration::error](#)

Copy item path

[source](#) ·

[-]

Expand description

All [Errors](#) generated in Barter-Integration.

Enums§

- [SocketError](#)
All socket IO related errors generated in barter-integration.